

Joystick Signal Processing and Trajectory Pipeline

Center-out length categorization task (Merchant Lab)
Task suite `centerTask` v8.26 and the EDA notebooks

This document describes the joystick signal path as it works in the version currently deployed: from the RZ2 analog-to-digital converter to the kinematic quantities computed in the exploratory data analysis (EDA) notebooks. Every transformation is stated first as a mathematical object, then located in the code (file, class or function, and the variable that carries it), and finally annotated with the parameters that control it.

The chain has two halves that live on different machines and in different languages. The *online* half (MATLAB, `centerTask/`) acquires the signal and writes raw trajectories with hardware-derived timestamps; it applies no filtering of any kind. The *offline* half (Python, EDA notebooks, section 1.1) cleans those trajectories and differentiates them. Because the online side never alters the shape of the signal, the two halves can be reasoned about independently, and the raw rows always remain in the exported file.

Contents

1	Notation and conventions	3
1.1	The whole pipeline on one page	3
2	Online chain: from the ADC to the trajectory file	4
2.1	System overview	4
2.2	Wire format and normalization	4
2.3	Time base: sample index to local time	5
2.3.1	The affine clock map	5
2.3.2	States of the map, and the refit algorithm	7
2.4	Runtime clock-skew guard	9
2.5	One clock per comparison	10
2.6	Gap accounting	11
2.7	Bounded drain and backlog dynamics	12
2.8	Flush before the loop starts	12
2.9	Transport selection	12
3	How the two joysticks are read	13
3.1	USB joystick: polled once per frame on Computer 2	13
3.2	RZ2 joystick, Computer 1 side: Synapse ring buffer and relay	14
3.3	RZ2 joystick, Computer 2 side: listen, drain, time, log	16
4	Screen mapping and hit testing	17
4.1	Affine map from joystick units to pixels	17
4.2	Circular acceptance regions	17

5	Trajectory buffer and export	18
5.1	Buffer layout	18
5.2	Three write paths, one timestamp rule	19
5.3	Epoch retagging at movement onset	19
5.4	Exports	19
6	Offline chain: the EDA signal-processing pipeline	20
6.1	Reading a session: source, profile and geometry	20
6.2	Pipeline order	21
6.3	Stage 1: Hampel screen	22
6.4	Stage 2: constant-jerk Kalman filter with RTS smoother	22
6.5	Stage 3: PCHIP resampling to a uniform grid	24
6.6	Stage 4: zero-phase Butterworth low-pass	26
6.7	Stage 5: speed, Savitzky–Golay acceleration, summary window	27
6.8	Data-quality flags	29
6.9	Offline clock check	30
7	Downstream analyses on the kinematics	31
7.1	Per-trial feature vector	31
7.2	Changes of mind from the initial heading	32
7.3	Where the timing columns enter the other analyses	34
8	Parameter reference: what to modify, where, and what it does	35
8.1	Online chain (MATLAB)	35
8.2	Offline chain (notebook, section 1.1 cell)	38
8.3	Recommended order of adjustment	40
9	Observations and open points in the current code	41

1 Notation and conventions

Symbol	Units	Meaning
f_s	Hz	RZ2 write rate of the joystick buffer. Seed value 24414.0625/26 = 939.0024 Hz, re-estimated online by RZ2ClockMap
a, b	s/sample, s	Slope and intercept of the affine clock map $\hat{t}(n) = b + a n$ (RZ2ClockMap.m)
ς_k	s	Clock skew measured at frame k : <code>sampleTime - trigTime</code> (ClockSkewMonitor.m)
$n_i \in \mathbb{N}$	samples	Monotone absolute sample index attached by the relay to sample i
$(v_{x,i}, v_{y,i}) \in [-1, 1]^2$	(dimensionless)	Normalized joystick deflection of sample i (normalization performed on Computer 1)
t_i	s	Local (Computer 2) <code>GetSecs</code> time assigned to sample i
(x_i, y_i)	px	Screen coordinates of sample i (origin top-left, y grows downward)
(x_c, y_c)	px	Screen centre
W, H	px	Screen width and height (1920, 1080)
s_x, s_y, o_y	-, -, px	RZ2 gain on each axis and vertical offset (<code>rz2ScaleX</code> , <code>rz2ScaleY</code> , <code>rz2OffsetY</code>)
T_0	s	<code>sessionT0</code> , the instant the first trial started
τ_i	ms	<code>Time_ms</code> = $\max(1000(t_i - T_0), 0)$
Δt_k	s	Irregular step $t_k - t_{k-1}$ in the offline chain
Δ	s	Uniform resampling step (0.008 s, i.e. 125 Hz)
c	cm/px	Pixel pitch, $c = 0.3108 \text{ mm/px}/10 = 0.03108 \text{ cm/px}$

Each axis is processed independently unless stated otherwise. Vectors are column vectors and $\mathbf{I}$ is the identity.

1.1 The whole pipeline on one page

Figure 1 is the map for everything that follows. Two quantities travel from left to right and each boundary transforms exactly one of them: the *value* of the joystick signal changes units (volts, then $[-1, 1]$, then pixels, then centimetres), and its *time* changes reference (ADC sample index, then Computer 2’s clock, then milliseconds since the first trial, then a uniform 125 Hz grid). Nothing on the online side changes the *shape* of the signal; every filter is offline, and it is reversible in the sense that the raw rows remain in the file.

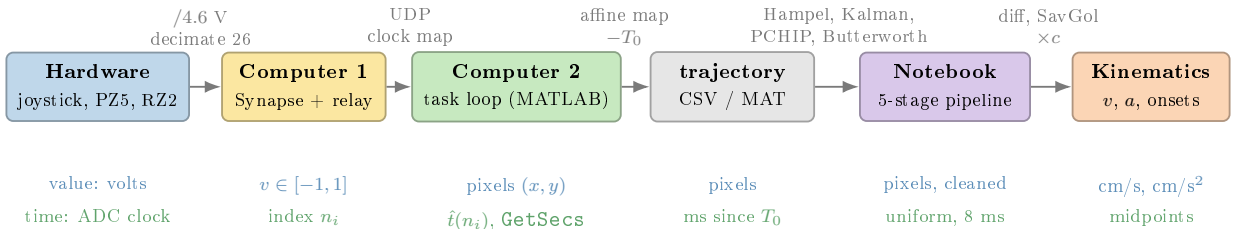


Figure 1: The signal’s two coordinates, value (blue lane) and time (green lane), and what each boundary does to them. Sections 2–3 cover the first three boxes, §5 the file, and §6–§7 the last two.

2 Online chain: from the ADC to the trajectory file

2.1 System overview

Figure 2 shows the two-machine link. Computer 1 (Windows) hosts TDT Synapse; the gizmos APICH1X and APICH2Y write the joystick's two analog channels into a ring buffer (`SerStore`) at f_s . A MATLAB relay (`StepJoystickRelay.m`, §3.2) reads new samples from that ring buffer through the write-index tag and pushes them over UDP to Computer 2 (Linux, Psychtoolbox), which only listens. The task loop on Computer 2 drains the socket once per rendered frame, renders the newest sample, and logs *every* sample to the trajectory buffer.

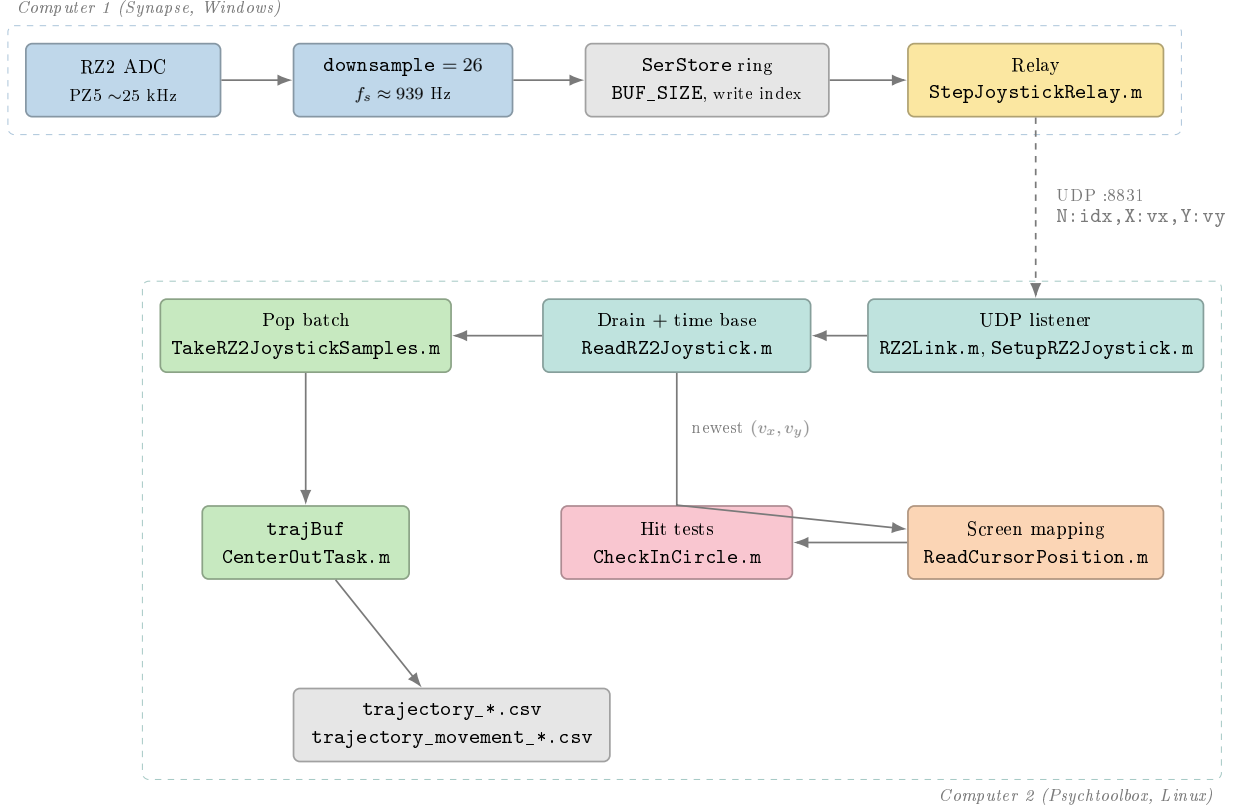


Figure 2: Acquisition and logging chain. Nothing in this chain filters the signal; it only timestamps, scales and stores. The drain feeds two consumers: the newest sample goes to the screen map and the hit tests, and the whole batch goes to the trajectory buffer.

2.2 Wire format and normalization

Each UDP datagram carries one relay cycle of samples, one text line per sample:

$$N:n_i, X:v_{x,i}, Y:v_{y,i} \backslash n \quad \text{with } v_{x,i}, v_{y,i} \in [-1, 1].$$

The normalization from ADC volts to $[-1, 1]$ is performed on Computer 1 (`JOY_RANGE = 4.6 V` in `InitJoystickRelay.m`; see §3.2), so Computer 2 never sees a voltage. Parsing is a single `sscanf` over the whole datagram followed by `reshapeRecords`, which keeps only $\lfloor \text{numel}/3 \rfloor$ complete records: a datagram torn mid-line yields fewer samples, never a fabricated one. Asking `sscanf` for a fixed size would instead zero-pad the final incomplete record, turning "N:101,X:0.6" into $[101; 0.6; 0]$, a plausible-looking sample that puts the cursor at $Y = 0$.

A relay that has not been updated sends one sample per datagram without an index ($X:v_x, Y:v_y$). Those datagrams are still accepted, so the two machines can be upgraded one at a time, and the samples are counted in `UserData.nLegacyFmt`.

2.3 Time base: sample index to local time

The relay attaches a monotone index n_i that counts ADC samples, so a sample's capture time is an affine function of its index. This is the key design decision of the online chain: a sample that sat in the receive queue for 200 ms still receives the time it was *captured*, not the time it was *read*. Figure 3 contrasts the two.

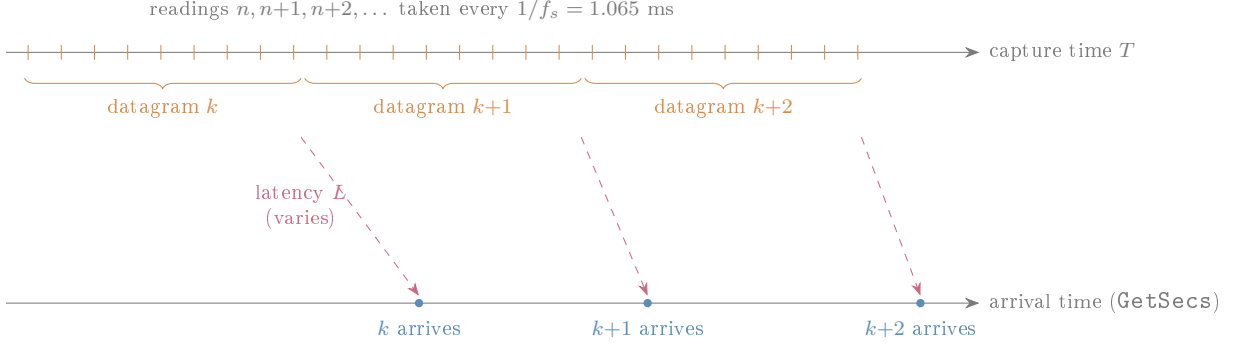


Figure 3: Capture time versus arrival time. Readings are taken at a fixed cadence (top); datagrams arrive later and irregularly (bottom). The serial number lets a reading recover its capture time no matter when its datagram arrived.

Two things can make the two clocks disagree. A constant *offset* is harmless for durations (both ends shift equally) and biases stimulus-to-response intervals by a constant. A *rate* error is not harmless: the disagreement grows every second and never stops growing. The mechanism below removes the rate error and bounds what is left of the offset.

2.3.1 The affine clock map

RZ2ClockMap.m implements

$$\hat{t}(n) = b + a n, \quad a \approx 1/f_s \text{ (seconds per sample)}, \quad (1)$$

with (a, b) refitted on every drain. Each drain d contributes one observation pair $(n^{(d)}, t^{(d)})$: the index of the *newest* sample in the drain and the `GetSecs` reading taken when the drain started. The newest sample is the least delayed one in the batch (everything older sat behind it in the queue), so it is the observation closest to the link's latency floor. Observations live in a ring of $L = \text{rz2ClockWindow} = 600$ entries; refits begin once at least `minObs` = 30 observations are held.

Slope: the seed first. Ordinary least squares over the window,

$$a_{\text{OLS}} = \frac{\sum_d (n^{(d)} - \bar{n})(t^{(d)} - \bar{t})}{\sum_d (n^{(d)} - \bar{n})^2}, \quad a = \text{clip}(a_{\text{OLS}}, a_0(1 - \delta), a_0(1 + \delta)), \quad a_0 = \frac{1}{f_s^{\text{seed}}}, \quad \delta = 0.01, \quad (2)$$

but only once the window *spans* at least `rz2ClockMinSpanSec` = 30 s of samples; until then $a = a_0$. OLS is unbiased for the slope even though every $t^{(d)}$ is contaminated by transport latency, provided the latency is stationary, but unbiased is not precise. For a window of W observations spanning S seconds with residual scatter σ_r ,

$$\text{se}(a) \approx \frac{\sigma_r}{\sigma_n \sqrt{W}}, \quad \sigma_n = \frac{f_s S}{\sqrt{12}}, \quad (3)$$

so a short window produces a slope that is mostly noise, while the seed is known to a few hundred parts per million from the rig itself. The window earns the right to change the slope only once its span makes the estimate better than the seed.

Queue-clear gate. The dangerous case is a *growing* backlog. Write the arrival time of the newest sample as its capture time plus a latency, $t_{\text{arr}}(n) = t_{\text{cap}}(n) + L(n)$. If L trends with n , the fitted slope is

$$a_{\text{OLS}} \approx \frac{1}{f_s} + \frac{dL}{dn}, \quad (4)$$

so the map attributes to a slower ADC what is really samples arriving later and later. Its output then tracks *arrivals* instead of captures, `trigTime` stays close to `sampleTime`, and the skew monitor of §2.4 sees nothing: the one failure it exists to catch becomes invisible. The estimator therefore refuses any observation from a drain that did not leave the receive queue empty. The call is `addObservation(idx, t, queueClear)` with `queueClear = (backlog == 0)`, and refusals are counted in `nObservationsRejected`. If the queue never clears, the map stops updating, the estimate holds where it was, and the skew grows until the monitor stops the session. Refusing to estimate is the correct response to data that cannot support the estimate.

Intercept: a low quantile over a short window. The intercept is anchored on the `rz2ClockOffsetQuantile = 0.10` quantile of the residuals of the most recent `rz2ClockOffsetWindow = 100` observations, rather than on the OLS intercept:

$$r_d = t^{(d)} - (b_{\text{OLS}} + a n^{(d)}), \quad b = b_{\text{OLS}} + Q_{0.10}(\{r_d\}_{\text{last } 100}). \quad (5)$$

b_{OLS} absorbs the *mean* transport latency; a low quantile tracks the fast path, which is the minimum-delay idea of clock synchronisation, without the fragility of a strict minimum (a single early packet or one corrupt index would drag it). The window is deliberately separate from the slope window and much shorter: the slope needs span, the offset needs to forget. A running minimum over a growing sample can only fall, which ratchets the offset backwards; a window that actually slides cannot ratchet. What survives in b is only the unobservable, constant floor latency of the link, which cancels in every within-trial difference. Figure 4 shows the two windows over the same stream.

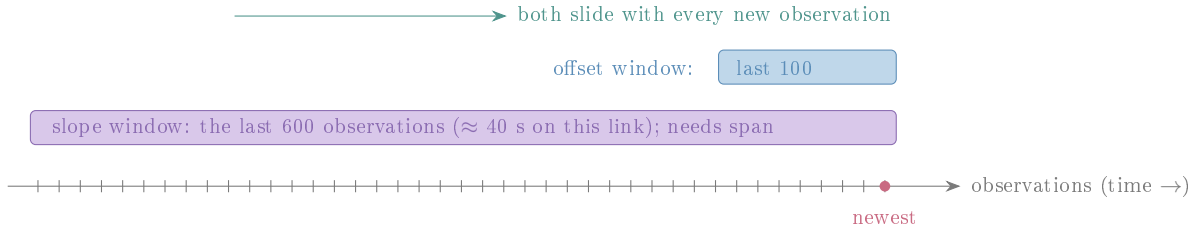


Figure 4: Two windows over the same stream of observations. The slope needs a long span and gets a long window; the offset needs to forget and gets a short one. Sharing one window length for both is what produced the ratchet described above.

Slew limit. No refit may move $\hat{t}$ at the newest index by more than `rz2ClockMaxSlewSec = 2 ms` (ten times that during the first `warmupFits = 100` fits, while the map is still settling). With x_{new} the newest index in the window the proposed move is

$$m = (b_{\text{new}} + a_{\text{new}}x_{\text{new}}) - (b_{\text{old}} + a_{\text{old}}x_{\text{new}});$$

if $|m| > s$ the intercept is pulled back so that the move equals $\pm s$ and the rest is applied over later fits. Since indices advance about 16 ms between refits and the line may move at most 2 ms against them, the emitted times cannot run backwards: monotonicity is a property of the design rather than a repair. Every fit slews, including the first, so the map can never snap from the single-pair warm-up anchor straight to the quantile line. Figure 5 contrasts a step with a slew.

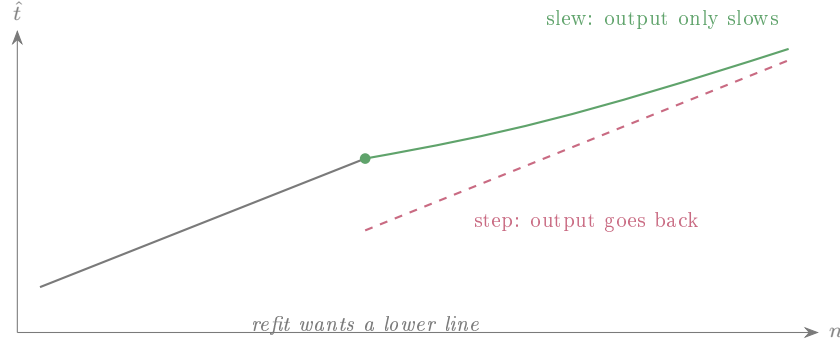


Figure 5: A refit that wants to move the line down. Applied as a step (dashed), the next timestamp is earlier than the last one handed out. Applied through the slew limit (green), the line bends down gradually while the index keeps advancing, and the output stays monotone.

Monotone guard. `indexToTime` still never emits a time earlier than the newest one already emitted, $\hat{t} \leftarrow \max(\hat{t}, t_{\text{last}})$, and counts such pulls in `nMonotoneClamped` and `worstMonotoneClampSec`. With the slew limit in place the count on a healthy session is zero or a handful; hundreds means the line stepped.

2.3.2 States of the map, and the refit algorithm

Figure 6 summarises the four states the map passes through, and Algorithm 1 is the refit itself. The algorithm runs once per accepted observation, after at least `minObs` observations are held. Lines 1–6 decide whether the window has earned the right to replace the seed slope: the span test on line 2 is Eq. (2)’s condition, and the clamp on line 3 bounds how far a fit may travel from the rig-confirmed seed. Lines 7–9 compute the residuals about the OLS line and lower the intercept onto their 10th percentile, which is Eq. (5). Lines 10–13 are the slew limit: the move is evaluated at the newest index, because that is where the next sample will be stamped, and the intercept absorbs whatever part of the move exceeds the limit. Line 14 commits both parameters together, so the map is never left with a new slope and an old intercept.

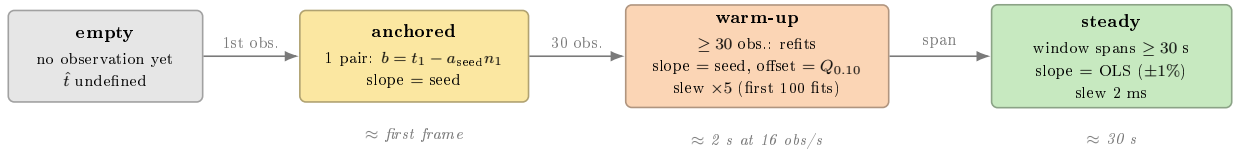


Figure 6: States of `RZ2ClockMap`. Timings are for this link, which yields about 16 accepted observations per second (roughly one drain in four arrives empty at 60 fps with the relay at 44 Hz).

Figure 7 shows what the estimator sees: a cloud of observations that all sit *above* the true line by their transport latency, from which OLS recovers the slope while the intercept is lowered onto the fast tail.

Implementation. `ReadRZ2Joystick.m` feeds one observation per drain and asks the map for every indexed sample’s time. The order of the four lines below is load-bearing: the backlog must be read *before* the timestamping, because it is what decides whether this drain’s pair may enter the estimator at all.

```
backlog = u.pending(); % RZ2Link.m; read BEFORE timestamping
rz2.clock.addObservation(iiNew(end), t2, backlog == 0); % newest index, t2, gate
tOut(haveIdx) = rz2.clock.indexToTime(idx(haveIdx)); % affine map, monotone-guarded
ud.batch = [ud.batch; [tOut, rows(:,2), rows(:,3), idx]]; % [time, vx, vy, absIdx]
```

Algorithm 1: `RZ2ClockMap.refit()`, run after each accepted observation.

Input: ring of accepted observations $(x_d, y_d) = (n^{(d)}, t^{(d)})$, count $n \geq \text{minObs}$

```

1  $x \leftarrow$  indices in window,  $y \leftarrow$  arrival times;  $\bar{x}, \bar{y}$  their means
2 if  $\max x - \min x \geq 30 \text{ s} \cdot f_s^{\text{seed}}$  then
3    $a \leftarrow \sum (x - \bar{x})(y - \bar{y}) / \sum (x - \bar{x})^2$ ; clamp  $a$  to  $[0.99, 1.01] / f_s^{\text{seed}}$ 
4 end
5 else
6    $a \leftarrow a_{\text{current}}$  // seed until the window has span
7 end
8  $b \leftarrow \bar{y} - a\bar{x}$ ;  $r \leftarrow y - (b + ax)$ 
9  $r_Q \leftarrow$  10th percentile of the most recent 100 entries of  $r$ 
10  $b_{\text{new}} \leftarrow b + r_Q$ 
11  $m \leftarrow (b_{\text{new}} + a x_{\text{newest}}) - (b_{\text{old}} + a_{\text{old}} x_{\text{newest}})$ 
12  $s \leftarrow 2 \text{ ms}$  ( $\times 5$  during the first 100 fits)
13 if  $|m| > s$  then
14    $b_{\text{new}} \leftarrow b_{\text{new}} - (m - \text{sign}(m) s)$ 
15 end
16  $(a_{\text{current}}, b_{\text{current}}) \leftarrow (a, b_{\text{new}})$ 

```

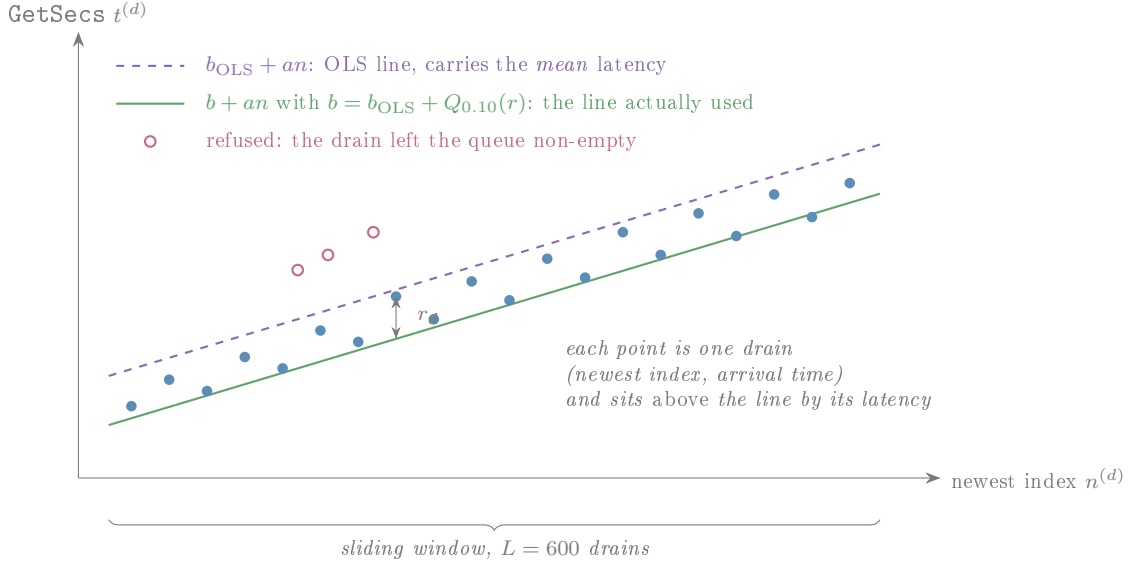


Figure 7: The affine clock map, Eq. (1). Observations sit above the true line by their transport latency; OLS recovers the slope, and the intercept is lowered to a low quantile of the residuals (10th percentile over the last 100 drains) so that only the floor latency remains as an offset. Drains that left the queue non-empty are refused, because a trending latency would bend the slope (Eq. (4)).

SetupRZ2Joystick.m constructs the map with the eight parameters of §8.1; [] in the fourth position leaves minObs at its default of 30:

```
'clock', RZ2ClockMap(OrigGet(orgParams, 'rz2SampleRateHz',      939.0024), ...
                    OrigGet(orgParams, 'rz2ClockWindow',        600), ...
                    OrigGet(orgParams, 'rz2ClockMaxRateDev',     0.01), ...
                    [], ...
                    OrigGet(orgParams, 'rz2ClockOffsetQuantile', 0.10), ...
                    OrigGet(orgParams, 'rz2ClockMaxSlewSec',     0.002), ...
                    OrigGet(orgParams, 'rz2ClockOffsetWindow',   100), ...
                    OrigGet(orgParams, 'rz2ClockMinSpanSec',     30))
```

Samples without an index. Legacy-format samples (no index) fall back to the estimate $t_j = t_1 + (t_2 - t_1)j/m_{\text{legacy}}$, spread evenly across the drain interval. They are counted in nLegacyFmt and exported with RZ2Idx = NaN. Because the map now lives in the GetSecs frame, legacy and indexed samples share one reference, so a legacy sample dropped into a batch can no longer produce a backward step of the accumulated divergence.

What the map does not fix. The floor one-way latency is invisible from Computer 2: a link uniformly 8 ms slow is indistinguishable from one that is not. It stays in b as a constant, cancels in every within-trial difference, and shifts every stimulus-to-response interval by that fixed amount. Measuring it needs a hardware loopback.

f_s is a seed, not a divisor. The constant only seeds a_0 and centres the clamp. A wrong seed costs a warm-up of tens of drains instead of a whole session. A Synapse downsample reset to 1 (write rate ~ 24 kHz) is far outside the $\pm 1\%$ clamp, so the slope clamps, the skew grows, and the monitor of §2.4 stops the run within a few frames. Confirm the value on the rig with getSamplingRates and the live downsample; do not infer it from another gizmo's displayed storage rate.

2.4 Runtime clock-skew guard

Every frame holds two readings of the same instant: sampleTime (GetSecs at the cursor read) and trigTime (the timestamp of the trajectory row that read produced, §5.2). On the mouse and USB-joystick paths they are the same number by construction. On the rz2adc path trigTime comes from the clock map, so

$$\varsigma_k = \text{sampleTime}_k - \text{trigTime}_k \quad (6)$$

is a direct per-frame measurement of the newest sample's age (queue depth plus any map error).

ClockSkewMonitor.m applies three rules over two statistics:

1. A throttled warning (at most one every warnPeriodSec = 5 s) when the *instantaneous* $|\varsigma_k| \geq \text{rz2SkewWarnSec} = 0.05$ s.
2. A latching trip when the *median of the last* rz2SkewWindowFrames = 90 frames (≈ 1.5 s) reaches rz2SkewAbortSec = 0.20 s. A median over 1.5 s is what a single stalled frame cannot fake and a persistent backlog cannot hide from; tripping on one frame killed a session for a transient stall.
3. A latching trip on a sustained run of *non-finite* skew readings. indexToTime returns NaN until the map anchors, and the map only anchors on a drain that left the queue clear. If the queue never clears for the whole session, every trigTime is NaN, ς_k is NaN every frame, and a monitor that

simply returned on a non-finite value would see nothing to warn about while every timestamp was invalid. The threshold is `nonFiniteAbortRun = 10 × windowLen = 900 frames` (≈ 15 s), long enough to cover ordinary startup before the first observation lands.

On a trip, `CenterOutTask.m` sets `exitFlag` and `abortedByClock`, so the end-of-session text distinguishes a clock fault from an operator abort, from a completed quota, and from a session that ran out of stimuli. Figure 8 shows why the median is the right statistic for the abort.

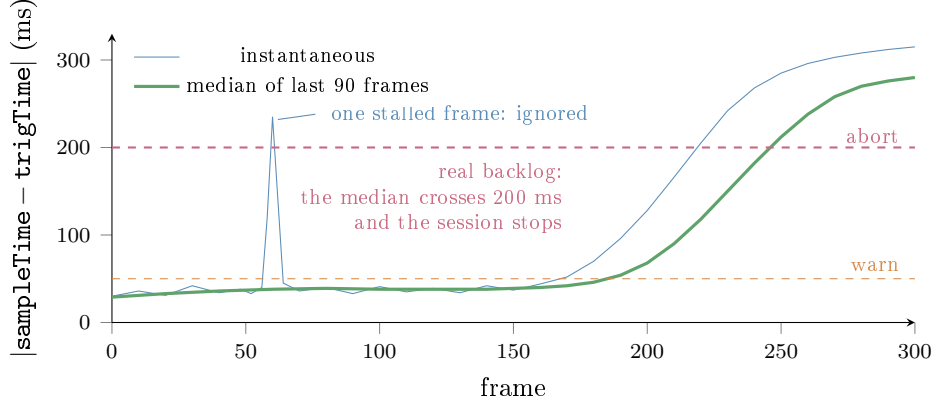


Figure 8: ClockSkewMonitor. A single spike (frame 60) never reaches the abort line because the median does not follow it; a genuine, growing backlog (from frame 170) does, and the session stops when the median crosses 200 ms.

2.5 One clock per comparison

With the sample clock corrected, one structural rule remains: the state machine must never compare a capture time against a wall-clock time inside a single expression. Every window anchored on a cursor event (`t.centerHold`, `t.leaveCenter`, `t.reachTarget`, `t.holdStart` in `CenterOutTask.m`, and `t.centerHold`, `t.targetHit` in `CenterInTask.m`) is tested against `trigTime`, not `this_time`.

Let T_ℓ and T_r be the capture times of leaving the centre and reaching the target, and L the transport latency. Then

$$\text{old: } \underbrace{T_r + L}_{\text{wall now}} \leq T_\ell + D \quad \Longleftrightarrow \quad T_r - T_\ell \leq D - L, \quad (7)$$

$$\text{new: } \underbrace{T_r}_{\text{trigTime}} \leq T_\ell + D \quad \Longleftrightarrow \quad T_r - T_\ell \leq D. \quad (8)$$

The latency cancels. The decision window is anchored on the stimulus flip (wall clock, from `Screen('AsyncFlipEnd')`'s `StimulusOnsetTime`) and tested against `trigTime` as well: that is the true reaction time up to the link's constant floor latency, instead of reaction time plus the queue. The honest side effect is that during a stall `trigTime` does not advance, so a hold takes the stall longer to confirm; without samples there is nothing to confirm it with. On mouse and USB input `trigTime` equals `sampleTime` and nothing changes. Figure 9 shows both rules on the same trial.

The two rules appear in the code as:

```
% CenterOutTask.m, EP.MOVEMENT
withinWindow = trigTime <= t.leaveCenter + maxExecutionTime;
% CenterOutTask.m, EP.HOLD
if trigTime > t.centerHold + holdTime && inCenterCircle
% CenterOutTask.m, EP.DECISION_TIME
if trigTime <= t.targetOnset + maxDecisionTime
```

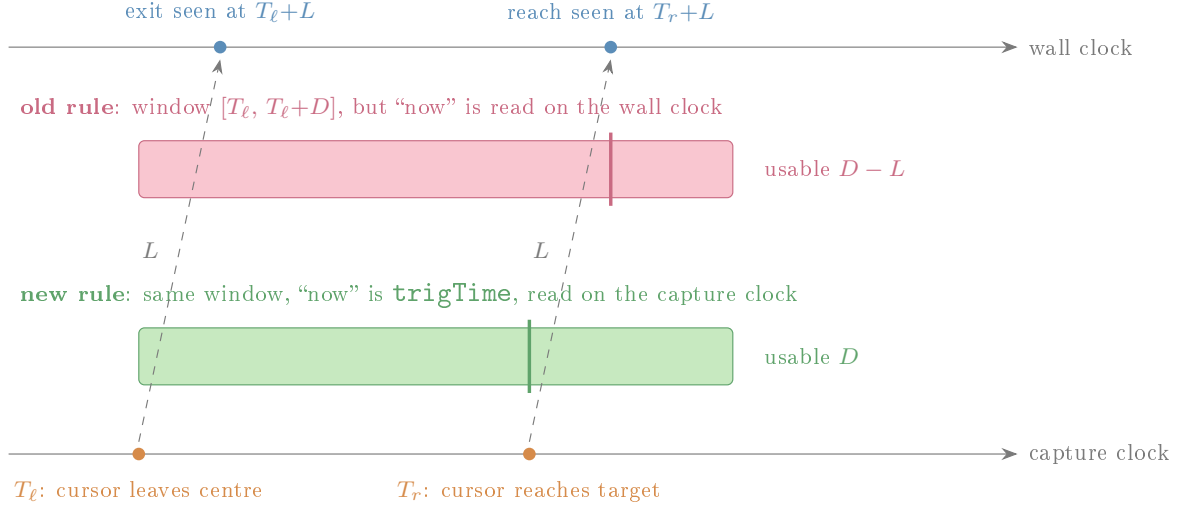


Figure 9: One clock per comparison. The window (the coloured band) is the same in both rules; only the “now” it is tested against changes, marked by the vertical bar. Reading “now” on the capture clock removes L from the inequality entirely.

The first line bounds the movement window: both ends are capture times, so the window is exactly `maxExecutionTime` long. The second confirms the centre hold from the capture clock and the position of the same row, so a hold is confirmed by samples rather than by elapsed wall time. The third anchors the decision window on `t.targetOnset`, which is the flip’s `StimulusOnsetTime` and therefore a wall-clock instant, and tests it against `trigTime`: that mixture is deliberate, because the stimulus really did appear at a wall-clock instant and the response really was observed on the capture clock, so their difference is the reaction time plus the link’s constant floor latency and nothing else.

Stimulus-driven windows that do not depend on the cursor (`cueDuration`, `barToTargetDelay`, the bar and ITI durations) stay on `this_time`: they measure how long something was on the screen, which is a wall-clock quantity.

The cached row. A frame that drained nothing writes a cached-position row (write path 2, §5.2). Under one clock per comparison, stamping it with `sampleTime` would make `trigTime` jump ahead of the stream by the link’s latency on every such frame. The row inherits the *previous row’s* time instead: “now” in capture time, when nothing new has arrived, is the time of the newest sample that exists. Before the first sample of the session there is no capture clock yet, so the wall clock stands in.

2.6 Gap accounting

Let $n^{(1)} < n^{(2)} < \dots$ be the indices seen across drains, including the seam with the previous drain (`ud.lastIdx`). The number of missing samples is

$$N_{\text{skipped}} = \sum_k \max(n^{(k+1)} - n^{(k)} - 1, 0), \quad (9)$$

accumulated in `ud.nSkipped`. Lost datagrams and relay recalibration jumps both appear here and are not distinguishable from Computer 2, so they are reported together. Gaps *between* datagrams of the same drain are counted too, not only the seam, because a single datagram lost in the middle of a busy frame is exactly the case a seam-only check would miss. A deliberate flush by `FlushRZ2Joystick.m` resets `lastIdx` to `NaN` so the discarded backlog is not counted as loss; it goes to `nFlushed` instead.

2.7 Bounded drain and backlog dynamics

Let M be the per-frame drain cap (`rz2MaxSamplesPerDrain` = 2048), f_{fr} the frame rate (60 Hz) and B a backlog that has accumulated in the socket. Each frame removes at most M samples while f_s new ones arrive per second, so the backlog after k frames is

$$B_k = \max\left(B_0 - k(M - f_s/f_{\text{fr}}), 0\right), \quad k^* = \left\lceil \frac{B_0}{M - f_s/f_{\text{fr}}} \right\rceil, \quad (10)$$

so a 97 100-sample burst clears in $k^* \approx 48$ frames (≈ 0.8 s) at $M = 2048$. An unbounded drain is what lets a backlog feed on itself: the longer the queue, the longer the frame that has to swallow it, which grows the queue further. The cap is checked *before* a datagram is read and a datagram is always consumed whole, so the true ceiling is M plus one relay cycle; capping mid-datagram would discard samples already taken off the socket, which is strictly worse than leaving them queued. A warning is emitted only after `rz2CapWarnFrames` = 60 consecutive capped frames (about one second), since transient capping after any loop stall is normal and self-clearing. The worst backlog seen and the number of capped frames are reported at teardown.

The receive buffer must hold the burst while it drains. At roughly 29 bytes per sample, `rz2InputBufferSize` = 4 194 304 bytes holds $\approx 145\,000$ samples (≈ 154 s at 939 Hz), comfortably above the largest burst seen.

2.8 Flush before the loop starts

`FlushRZ2Joystick.m` discards whatever queued between socket open and trial-loop start. The setup steps and the modal recording checklist (`ConfirmRecordingLink.m`) can take many seconds, i.e. thousands of samples, and those samples describe a period the task was not running: they are backlog, not data. The function resets `batch`, `lastIdx`, `capConsec` and `lastDrainTime`, and returns the number of datagrams (or bytes, on the legacy transport) discarded.

The clock map is deliberately left alone. Its observations are (index, arrival) pairs, and discarding queued datagrams removes future observations without invalidating any past one; re-anchoring here would introduce exactly the discontinuity the flush exists to avoid, and the session stays on one timeline. A stale first observation is no longer fatal in any case (it is one of L and is eventually refitted away), but flushing still avoids seconds of capped frames at the start of the session.

2.9 Transport selection

`RZ2Link.m` owns the socket and hides which transport is in use behind one API: `readOne()`, `pending()`, `flush()`, `close()`, `describe()`. It tries three transports in order and keeps the first that opens:

1. `java.nio.channels.DatagramChannel`, **non-blocking** (`mode` = 'java'). This is what the rig uses. A non-blocking channel has no helper thread: `receive()` returns immediately with a datagram if the operating system has one and `null` if it does not, so the drain loop sees the OS queue directly, at frame rate. It ships with every JRE MATLAB has bundled. `pending()` cannot report a depth here and returns 0 when the last `readOne` found the queue empty and 1 otherwise, which is exactly the boolean the clock map's gate needs.
2. `udpport` (MATLAB R2019b and later), `pending()` = datagrams available.
3. **Legacy** `udp()`, with `InputBufferSize` and `InputDatagramPacketSize` applied; `pending()` = bytes available.

The rig's Computer 2 runs MATLAB R2016b, so `udpport` is unavailable and the legacy object was the transport until the NIO channel replaced it. The legacy object delivers datagrams through

a Java helper thread that fills `BytesAvailable` on its own schedule, and on this rig that schedule is coarse: with the relay sending every ≈ 23 ms, a third of the 67 ms drain windows came back empty, which is impossible under direct delivery (three relay cycles fit in one window). Datagrams were being handed over in clumps, and waiting for the next clump was most of the measured sample age. A transient Java exception on the non-blocking receive (for example an ICMP port-unreachable when the relay bounces) is counted in `nReadErrors` and treated as an empty read, because it is recoverable on the next frame and must not abort a session. The mode actually opened is printed at startup and again at teardown.

3 How the two joysticks are read

The task accepts three input sources, selected by `orgParams.inputSource ('joystick' | 'mouse' | 'rz2adc')`. The two joysticks differ in every respect that matters for a trajectory: where they are sampled, at what rate, who timestamps them, and how many rows per frame they produce. Table 1 summarises; the subsections give the mechanics.

	USB joystick ('joystick')	RZ2 analog joystick ('rz2adc')
Transducer read by	Computer 2, <code>vrjoystick(1)</code> (Simulink 3D Animation)	PZ5 amplifier $\rightarrow$ RZ2 processor, TDT Synapse on Computer 1
Sample rate	Two reads per rendered frame (base + one oversample 8 ms later), ≈ 120 Hz; HID native ≈ 125 Hz	$f_s \approx 939$ Hz per axis, independent of the frame rate
Native units	Axes already in $[-1, 1]$ (driver)	Volts, normalised to $[-1, 1]$ by <code>JOY_RANGE = 4.6 V</code> on Computer 1
Timestamp	<code>GetSecs</code> at the read (<code>sampleTime</code>); exact by construction	Relay sample index n_i mapped through <code>RZ2ClockMap</code> into the <code>GetSecs</code> frame
Rows per frame	2	All samples drained that frame, typically 15–17
Latency	USB polling interval, sub-frame (4–10 ms)	Relay period + UDP + queue depth; measured per frame as ς_k
Clocks in play	one	two, reconciled by <code>RZ2ClockMap</code> (§2.3)
Can drift	no	only if the rate were a fixed constant, which it is not
Runtime guard	none needed	<code>ClockSkewMonitor</code> (§2.4)
Provenance column	<code>RZ2Idx = NaN</code>	<code>RZ2Idx = n_i</code>
Gain / offset	<code>joyGain (-1.3)</code> , <code>pointer_offset</code>	<code>rz2ScaleX</code> , <code>rz2ScaleY (1, 1)</code> , <code>rz2OffsetY (0)</code>

Table 1: The two joystick paths side by side.

3.1 USB joystick: polled once per frame on Computer 2

`SetupJoystick.m` opens the device once, `joy = vrjoystick(1)`, and fails with an explicit error if index 1 is not a recognised joystick. Each frame `ReadCursorPosition.m` does:

```
axesVals = read(joy); % [ax1, ax2, ...] in [-1, 1]
x = xCenter + screenXpixels * axesVals(1, 1) * joyGain;
y = yCenter + pointer_offset + screenYpixels * axesVals(1, 2) * joyGain;
```

so the sampling process is the frame loop itself: the joystick is a function $\mathbf{u}(t)$ observed at the frame instants t_k , with $t_{k+1} - t_k \approx 1/60$ s and jitter equal to the loop's own jitter. There is no buffer, no index and no queue; the timestamp written to `trajBuf` is the same `GetSecs` the state machine uses, so the skew of §2.4 is identically zero and the monitor is not run on this path.

The oversample loop (`moveOversample = 1`, `moveOversampleDt = 0.008 s`) takes one extra read while the asynchronous flip is pending. It is skipped entirely on the `rz2adc` path, where the batch write already logs every sample the relay delivered and a second read would only re-drain an already-drained socket. The 8 ms spacing is this rig’s measured HID update interval (≈ 125 Hz), so two reads per frame is the most the device can deliver without duplicating cached values; the effective rate is therefore ≈ 120 samples/s, close to the 125 Hz grid of §6, and the Savitzky–Golay window of 11 grid samples spans roughly 11 real reads. One extra read is used rather than several because at ≈ 8 ms per sample, three extra reads would span 24 ms, longer than a 16.7 ms frame, and the burst would run past the vsync it is supposed to be free inside of.

On this path `trigTime` is literally `sampleTime`, so every comparison in the state machine was already on one clock and §2.5 changes nothing; `Time_ms` is non-decreasing by construction, and there is no rate to get wrong, no anchor, no window and no monitor.

3.2 RZ2 joystick, Computer 1 side: Synapse ring buffer and relay

Signal path in Synapse. The joystick’s two analog outputs enter the PZ5, which streams to the RZ2 at the TDT base rate $f_{\text{base}} = 24414.0625$ Hz. Two `APIStreamer1Ch.rcx` gizmos, `APICH1X` and `APICH2Y`, decimate by `downsample = D = 26` and write into a `SerStore` ring buffer of $B = \text{BUF_SIZE} = 100\,000$ samples each:

$$f_s = \frac{f_{\text{base}}}{D} = \frac{24414.0625}{26} = 939.0024 \text{ Hz}, \quad \text{ring capacity} = \frac{B}{f_s} \approx 106.5 \text{ s}. \quad (11)$$

Each gizmo exposes a write-index tag (surfacing through SynapseAPI under the literal name ‘???’, the tag having reached the API without a resolved label), the position $W \in [0, B)$ of the next sample to be written.

Relay cycle. The relay (`StepJoystickRelay.m`) is stepped from the `categTaskCommunication` GUI loop and is a no-op until `RELAY_PERIOD = 1/169.4 s` has elapsed. In write-index mode (`WRITE_IDX_TAG_X/Y` non-empty, the configuration on this rig) one cycle is:

1. Read the write heads W_x, W_y through `getParameterValue`, reduced modulo B .
2. Compute the signed circular distance from the read cursor C to the head,

$$\Delta(W, C) = ((W - C) \bmod B) - B \lfloor (W - C) \bmod B > B/2 \rfloor, \quad (12)$$

(`circDelta`: the representative of $W - C$ in $(-B/2, B/2]$). A negative value means the head was read behind the cursor, which is jitter or pacing that outran the writer; it is counted in `nHeadBehind` and clamped to 0. The unsigned version, `circDist`, discards the sign on purpose for its tolerance check, which is why the signed variant had to be introduced separately.

3. Read $n = \min(\Delta_x, \Delta_y, \text{MAX_READ_PER_CYCLE} = 4096)$ samples from each ring at offset C , in two pieces if the window wraps past B (`readBufWindow`), and advance $C \leftarrow (C + n) \bmod B$.
4. Normalise and clip, $v = \text{clip}(\text{raw}/\text{JOY_RANGE}, -1, 1)$, with `JOY_RANGE = 4.6 V`.
5. Attach absolute indices $n_i = N_{\text{abs}}, N_{\text{abs}} + 1, \dots$ (the running count `absIdx`, never reduced modulo B) and send them as datagrams of at most `MAX_SAMPLES_PER_DGRAM = 17` lines, formatted column-major by a single `sprintf` with `'N:%.0f,X:%.4f,Y:%.4f\n'`.
6. $N_{\text{abs}} \leftarrow N_{\text{abs}} + n_{\text{done}}$, where n_{done} is how far the read cursors *actually* advanced this cycle: n when both buffer reads succeeded, 0 if either threw. Advancing by n unconditionally, outside the `try`, while the cursors advanced inside it, meant a failed read re-sent the same samples under new indices.

Healthy steady state is $n \approx f_s/169.4 \approx 5.5$ samples per cycle, i.e. about 169 datagrams per second on the wire (two per cycle since the per-datagram cap went to 17). After any stall on Computer 1 the backlog is read in bursts of up to 4096 samples, split into datagrams of 17. Figure 10 shows the two cursors on the ring.

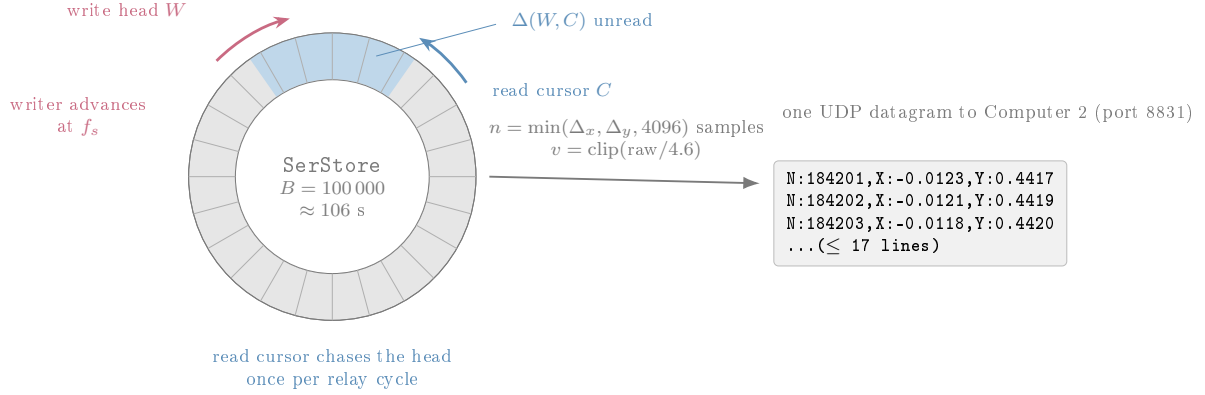


Figure 10: Relay cycle on Computer 1: both cursors move counter-clockwise, the writer at f_s , the read cursor in jumps of n once per cycle. The shaded arc is what is still unread, and it is what gets normalised, indexed with the running `absIdx`, and pushed over UDP.

Datagram size. Both machines use the legacy `udp()` object on the send side, whose `OutputDatagramPacketSize` defaults to 512 bytes and truncates anything larger. At 26–27 bytes per sample, 40 samples per datagram is ≈ 630 bytes and every datagram was cut at byte 512. The cap is now `MAX_SAMPLES_PER_DGRAM = 17` (17×30 bytes worst case = $510 < 512$), and the packet-size properties on both sides were raised to 8192 as a second lock.

Cost per SynapseAPI call. Every `getParameterValue` is an HTTP round trip of ≈ 5 ms regardless of payload, so four calls per cycle pin the loop at ≈ 44 Hz against a 169.4 Hz target. The Y write index is therefore read only every `IDX_Y_CHECK_EVERY = 10` cycles: X and Y advance in lockstep by construction, the relay already takes their minimum, and a check that finds them more than `IDX_Y_TOL = 2` samples apart warns once and reverts to reading both every cycle. Three calls instead of four is ≈ 17 ms per cycle. Getting to two would need X and Y interleaved in one `.rcx` buffer, which is a circuit change.

Legacy fallback, and the startup check that makes it reachable. If the write-index tags cannot be read, the relay paces itself by time, $n = \text{round}(\text{READ_FS} \cdot \Delta t_{\text{cycle}})$ with $\text{READ_FS} = \text{N_READ} \times \text{RELAY_HZ} = 6 \times 169.4 = 1016.4$ Hz, and to re-synchronise every `RECAL_INTERVAL_SEC = 10` s by the diff-peak method: two snapshots of a `RECAL_WINDOW = 6000`-sample window separated by `CALIBRATION_WAIT_SEC = 0.15` s, with the head located at $\arg \max_j (|d_2 - d_1| * \frac{1}{100} \mathbf{1}_{100})_j$ (`ActiveIndexFromSnapshots.m`).

How this path is entered is the part that changed. Reading a tag is not what detects a bad tag: SynapseAPI's `getParameterValue` and `getParameterValues` do not throw for a nonexistent name, so `idx0k` inside `readWriteIndex()` can never go false from a wrong `WRITE_IDX_TAG_X/Y`, `widxFellBack` never fires, and the relay would anchor to whatever the call returned and then read zero new samples for the rest of the session without a warning. `InitJoystickRelay.m` therefore checks existence once, at startup, immediately after the two tag names are set: `getParameterNames` is called on `APICH1X` and `APICH2Y` and each name is looked up case-insensitively in the list it returns. If either is absent, a warning names the missing tag and its gizmo, both fields are reset to the empty string, and the session runs on the legacy path, which is exactly what the empty-string convention already meant. `getParameterNames` rather than `getParameterInfo` because `VerifyWidxWidy.m` found

that `getParameterInfo` can fail on tags that do exist, which makes it a diagnostic and not a test; `getParameterNames` is what `ProbeJoystickBuffers.m` and `VerifyWidxWidy.m` already use to prove a tag is real.

The check is at startup and not per cycle on purpose. A `getParameterNames` round trip inside `readWriteIndex()` would put a fourth HTTP call back into a loop whose cost is measured and logged, to catch a tag that disappears mid-session, which is not a failure this rig has produced. What remains true of the legacy path is that `WRITE_FS = 1017`, `READ_FS` and the recalibration constants were tuned against a write rate that was never real, so the fallback is now reachable but still mis-tuned; it is a loud degradation rather than a silent freeze, not a second correct mode. On this rig the question is hypothetical: `WRITE_IDX_TAG_X/Y` are '???' and have been readable since 2026-08-25, so the check exists for the next repointing.

WRITE_FS = 1017 is still active in the diagnostics. It no longer drives the read, but it feeds two things an operator is told to watch: the “ $\sim X$ ms behind head” line of the 5-second log, which divides the backlog by 1017 and therefore reads 7.7 % low against 939 Hz, and `VerifyWidxWidy.m`’s `ASSUMED_FS = 1017`, which would report a 7.7 % deviation from a link that is behaving perfectly. The maintainers’ stated plan is not to paste 939 in but to have `InitJoystickRelay.m` measure the rate at relay startup, as Computer 2 now does, keeping the constant as seed and sanity bound; that yields two independent estimates of the same quantity, one per machine, whose disagreement is itself a diagnostic.

3.3 RZ2 joystick, Computer 2 side: listen, drain, time, log

Computer 2 only listens. `SetupRZ2Joystick.m` opens port 8831 through `RZ2Link.m` (§2.9), which selects the transport at startup and prints which one it opened. `UserData` lives on the `RZ2Link` handle in every mode, so `rz2.port.UserData` is unchanged for the rest of the code, and a channel left open by a crashed session is closed on the next open. Per frame:

1. `ReadCursorPosition.m` calls `ReadRZ2Joystick.m`, which drains up to M samples (§2.7), feeds the clock map one observation, times every sample through Eq. (1), appends $[t_i, v_{x,i}, v_{y,i}, n_i]$ to `UserData.batch`, and returns only the newest (v_x, v_y) for rendering and hit-testing.
2. `CenterOutTask.m` calls `TakeRZ2JoystickSamples.m`, which pops the whole batch and clears it, and writes one trajectory row per sample (§5.2). The pop is a separate function from the drain so the socket is drained exactly once per frame (inside the render read) while the full batch is still available for logging.
3. The state machine acts on the newest row; the skew monitor compares its timestamp to `sampleTime` (§2.4).

Figure 11 compares the two input paths and Figure 12 shows what one frame contributes on each.

Resolution. The relay writes four decimals in $[-1, 1]$, so the smallest representable step is 10^{-4} in normalised units, i.e. 4.6×10^{-4} V. Through Eq. (13) this maps to $Ws_x \times 10^{-4} = 0.19$ px horizontally and $Hs_y \times 10^{-4} = 0.11$ px vertically at unit gain: the wire format is not the limiting quantisation. The USB path’s resolution is whatever the HID driver reports (commonly 8–10 bits per axis, i.e. 5–20 px at $|\text{joyGain}| = 1.3$), which is one reason the Hampel stage’s $\text{MAD} = 0$ guard (§6.3) fires more often on that path.

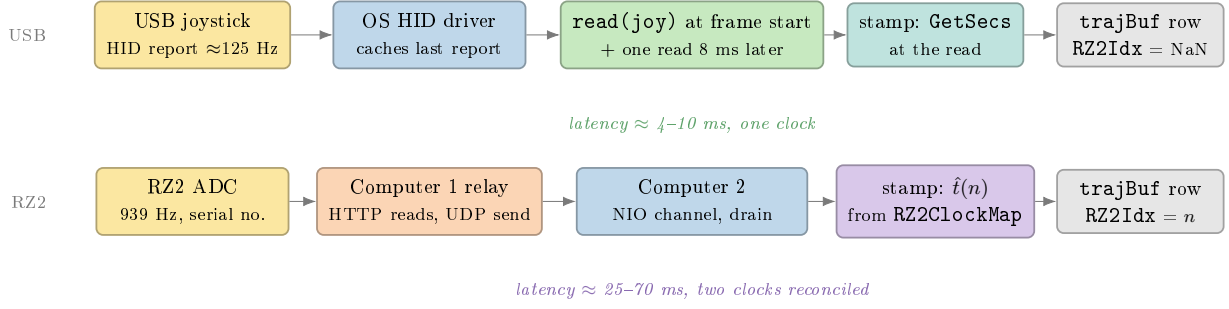


Figure 11: The two input paths side by side. The USB path has one machine and one clock; the stamp is the wall clock at the moment of the read. The RZ2 path has two machines, a serial number per sample, and an estimator that maps serial numbers back onto Computer 2’s clock. The point of the exercise is the last row of Table 1: the RZ2 path is harder because its samples are on the same clock as the neural recording.

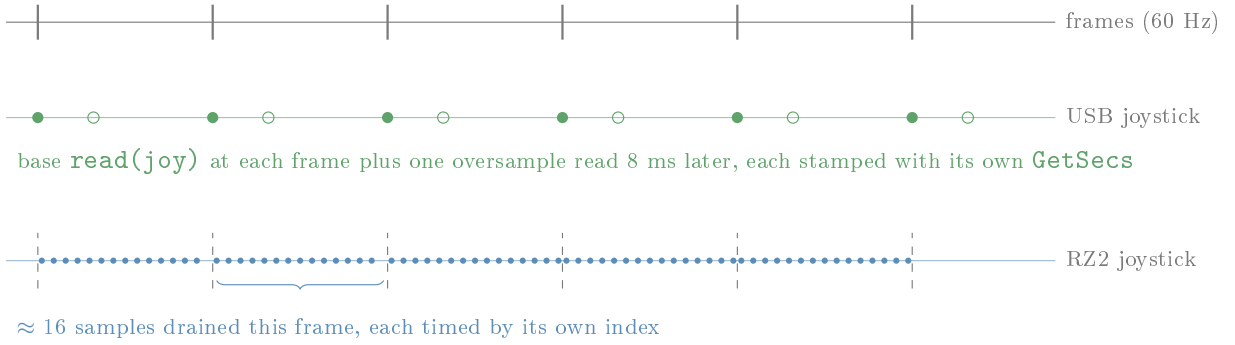


Figure 12: What one frame contributes to the trajectory on each path.

4 Screen mapping and hit testing

4.1 Affine map from joystick units to pixels

For the `rz2adc` input source (`ReadCursorPosition.m` and the batch loop in `CenterOutTask.m`):

$$x_i = x_c + W s_x v_{x,i}, \quad y_i = y_c + o_y + H s_y v_{y,i}. \quad (13)$$

The map is affine and axis-separable; s_x, s_y carry both magnitude (pixels per unit deflection, as a fraction of the screen extent) and sign (axis inversion). For the USB joystick the same form is used with $s_x = s_y = \text{joyGain}$ (default -1.3) and $o_y = \text{pointer_offset}$. Mouse mode reads pixels directly.

Because Eq. (13) is applied identically to the rendered sample and to every logged batch sample, the CSV columns `X_px` and `Y_px` are exactly the pixel coordinates the state machine saw. The inverse map recovers joystick units offline if needed: $v_x = (x - x_c)/(W s_x)$, $v_y = (y - y_c - o_y)/(H s_y)$.

4.2 Circular acceptance regions

Both the centre hold window and the targets are drawn as discs and tested as discs. Given a Psychtoolbox rect $[x_1, y_1, x_2, y_2]$,

$$c_x = \frac{x_1 + x_2}{2}, \quad c_y = \frac{y_1 + y_2}{2}, \quad r = \frac{x_2 - x_1}{2}, \quad \text{inside} \iff (x - c_x)^2 + (y - c_y)^2 \leq r^2. \quad (14)$$

`CheckInCircle.m` takes (c_x, c_y) and the left and right edges; `CheckInTargetCenterOut.m` takes the full rect and returns false for an unplaced (all-zero) rect. Both are exact, with no dependence on frame rate or hardware.

Figure 13 shows the geometry and how the epoch tags follow from these tests. The hit tests on each sample are what convert a continuous path into the `DECISION_TIME` / `MOVEMENT` / `TARGET_HOLD` labels in the trajectory file. The `MOVEMENT` epoch is the one that measures the *execution time*, $t.\text{reachTarget} - t.\text{leaveCenter}$, which is what `ExecutionTime_s` carries in `trial_data_*.csv`; the figure is labelled with that name, and `MOVEMENT` (code 7) is the identifier the code and the Epoch column use for it.

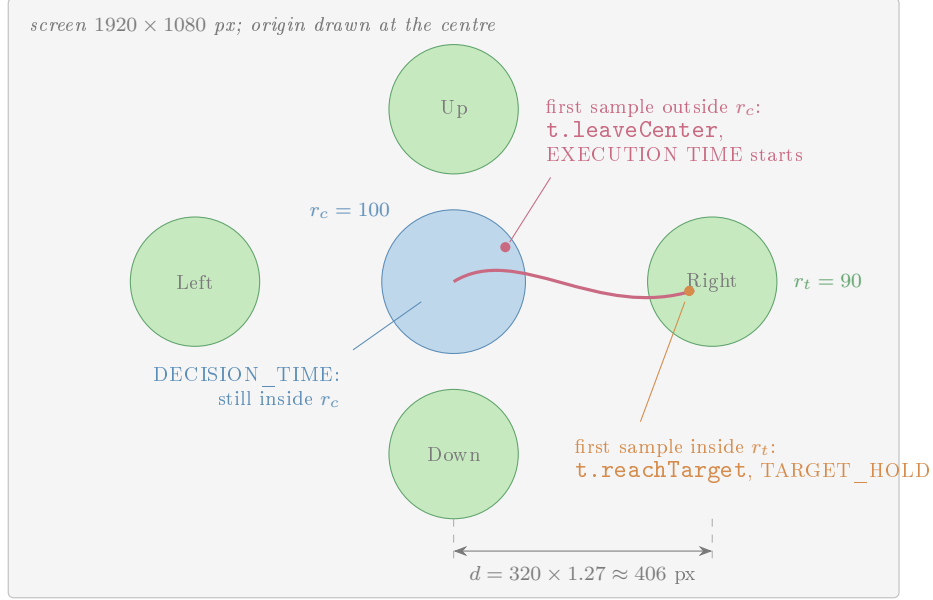


Figure 13: Geometry used by the hit tests, Eq. (14), and how the epochs follow from them. The interval that opens at $t.\text{leaveCenter}$ and closes at $t.\text{reachTarget}$ is the execution time; its epoch code is `MOVEMENT = 7`. Distances come from the session’s own `params_<runTag>.mat` when the notebook reads it (§6.1); the task reads its own values from the console.

5 Trajectory buffer and export

5.1 Buffer layout

Every logged sample is one row of `trajBuf` $\in \mathbb{R}^{N \times 9}$ in `CenterOutTask.m` (and $N \times 7$ in `CenterInTask.m`, which has no `Block` or `TrialNumInBlock` columns):

$$[\text{TrialNum}, \tau, x, y, \text{Epoch}, \text{Block}, \text{TrialNumInBlock}, \text{Attempt}, \text{RZ2Idx}], \quad (15)$$

with

$$\tau = \max(1000(\hat{t}_i - T_0), 0). \quad (16)$$

`RZ2Idx` is the relay’s absolute sample index n_i for rows the RZ2 batch wrote, and `NaN` for every other row (USB, mouse, oversample, empty-batch cached rows, legacy-format samples). It is the row’s provenance flag: `NaN` means the time is estimated rather than index-derived, and it lets an offline analysis re-derive timing under a different clock model without re-running anything (§6.9). The clamp at 0 exists because the very first drained batch of a session can carry times slightly earlier than T_0 , and it does so on either path: an indexed sample is timed by the clock map, a legacy index-less sample is interpolated from `lastDrainTime`, and both are readings of `GetSecs` on a link that was already streaming from Computer 1 before Computer 2 stamped T_0 . The clamp writes those few rows as exactly 0 rather than letting a trajectory open with negative times.

The buffer is preallocated at `trajChunk = 200 000` rows and grows in chunks of that size (or by the batch size if larger), so no per-row reallocation happens inside the frame loop.

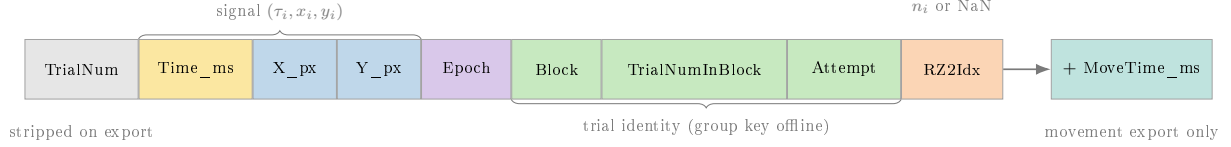


Figure 14: Row layout of `trajBuf` (`CenterOutTask.m`) and its two exports. `TrajectoryColumnNames.m` recognises the 8- and 6-column stripped layouts (with `RZ2Idx`) and the older 7- and 5-column ones.

5.2 Three write paths, one timestamp rule

Per frame the loop writes rows through one of three paths, all producing τ the same way, that is, at the instant the row's own (x, y) was read:

1. **rz2adc with a non-empty batch:** one row per drained sample, with x, y from Eq. (13) applied to that sample's (v_x, v_y) , τ from Eq. (1), and `RZ2Idx` = n_i .
2. **rz2adc with an empty batch** (relay stall): one row with the cached last position and the *previous row's* τ , so `trajN` still advances exactly once per frame and the column stays on one clock (§2.5); `RZ2Idx` = NaN marks it as cached.
3. **USB joystick or mouse:** one row per frame, plus `moveOversample` rows taken `moveOversampleDt` apart while the asynchronous flip is pending; `RZ2Idx` = NaN.

The state machine then acts on the newest row:

```
trigRowIdx = trajN;
trigTime   = sessionT0 + trajBuf(trigRowIdx, 2) / 1000;
```

Reading the time back *out of the row*, rather than copying it from `sampleTime`, is what guarantees that `t.leaveCenter` and `t.reachTarget` refer to the same sample the hit test used, and therefore that `MoveTime_ms` = 0 lands exactly on the leave-centre sample downstream. Both `trigRowIdx` and `trigTime` are captured before the render block, because the oversample loop reassigns `sampleTime` and advances `trajN`: by the time the state machine runs, those would refer to the last extra read rather than to the sample that drove the decision.

5.3 Epoch retagging at movement onset

When the cursor first leaves the centre after target onset, the rows from the triggering sample to the current end of the buffer are retagged:

```
trajBuf(trigRowIdx:trajN, 5) = EP.MOVEMENT.Value;
```

so the first row carrying the `MOVEMENT` code (7) is the sample whose hit test returned “outside centre”. Without this the epoch label lands one frame late, the first `MOVEMENT` row is the next frame's, and `MoveTime_ms` = 0 sits a frame behind `t.leaveCenter`. Retagging also puts the real first point of the movement into the export, which it was otherwise missing. Epoch codes used offline: `DECISION_TIME` = 6, `MOVEMENT` = 7, `TARGET_HOLD` = 8.

5.4 Exports

`SaveTrajectory.m` writes the whole buffer (all epochs) with the `TrialNum` column removed. The movement export (`SaveMovementTrajectory.m`) keeps rows with `Epoch` $\in$ `movementExportEpochs` = {6, 7, 8} and appends

$$\text{MoveTime_ms}_i = \tau_i - \min_{j \in g(i)} \tau_j, \quad (17)$$

where $g(i)$ is the set of rows sharing the identity columns (Block, TrialNumInBlock, Attempt) with row i . The minimum, rather than the first row, is used because batch timestamps can land slightly out of row order, which would otherwise make the first record's `MoveTime_ms` negative and shift the whole trial. The identity columns are those *between* Epoch and RZ2Idx: including RZ2Idx in the group key would make every row its own group and every `MoveTime_ms` exactly zero, silently, since a column of zeros is a perfectly well-formed column.

`Time_ms` is left untouched in both files, so rows can be joined across them. Both exporters call `ReportTimeMonotonicity.m`, which prints the number and worst size of backward steps in `Time_ms` and otherwise stays silent. It reports and does not repair: reordering or clamping there would hide a live instrumentation fault behind a tidy-looking file, so a file with a warning is one whose intervals must not be trusted where the steps occur. Both exporters also refuse to guess a layout: the caller declares its own literal column count (`expectedRawCols`, 9 for `CenterOutTask.m` and 7 for `CenterInTask.m`), and a mismatch is treated as a save failure and recorded by `WriteSaveFailureMarker.m` rather than written out.

Alongside the trajectories, `CenterConsole.m` writes `params_<runTag>.mat` *before* the task starts: a snapshot of `runTag`, session and subject identifiers, engine name, timestamp, and the full merged `orgParams` struct with the non-serializable `handles` field removed. A crash mid-session therefore still leaves a record of what was configured, and the offline geometry of §6.1 is read from it.

6 Offline chain: the EDA signal-processing pipeline

All offline processing lives in the section 1.1 cell of the single-session notebook (`EDA_SINGLE_SESSION_...ipynb`); the multi-session notebook carries a copy of the same definitions in its section 0.2 cell, with the per-trial figure drawer and `run()` left behind. The classes involved are `ScreenGeometry`, `InputSourceDetector`, `HampelScreen`, `KalmanTrajectorySmoother`, `ButterworthLowpass`, `TrajectoryProcessor` and `TrajectoryDataset`, plus `OfflineClockFit`, which is in the single-session notebook only (§6.9).

6.1 Reading a session: source, profile and geometry

Before any filter runs, `TrajectoryDataset.__init__` decides three things about the file it has just read.

Input source. `InputSourceDetector.detect` classifies each trajectory file as `rz2adc`, `usb` or `unknown`:

$$\text{source} = \begin{cases} \text{override} & \text{if the run tag matches a key of INPUT_SOURCE_OVERRIDES,} \\ \text{rz2adc} & \text{else if RZ2Idx has any finite value,} \\ \text{rz2adc} & \text{else if } \text{median}_k(\Delta\tau_k) < \text{RZ2_MAX_MEDIAN_STEP_MS} = 3 \text{ ms,} \\ \text{usb} & \text{else if that median is finite,} \\ \text{unknown} & \text{otherwise.} \end{cases} \quad (18)$$

The median step is taken within trials (grouped by Block, TrialNumInBlock, Attempt) over positive differences only, so it is not contaminated by the jump between trials. The RZ2 path delivers ≈ 1.06 ms steps and the USB path ≈ 8 ms, so the two are separated by a factor of eight; mouse sessions also land in `usb`. The detection is printed per file and appears in every figure title.

Pipeline profile. The source selects a `PipelineProfile` from `PIPELINE_PROFILES`, which carries the Hampel half-window and threshold, the Kalman measurement and jerk noise, the χ^2 gate, and one boolean, `drop_unindexed_rows`. Both profiles currently start from the same

filter values, so nothing changes until a profile is edited; the mechanism exists so that a per-source change can be made in one place rather than by branching inside the processor. `TrajectoryProcessor.for_profile` is the only constructor the analysis cells use.

Un-indexed rows. On a session detected as `rz2adc`, rows whose `RZ2Idx` is `NaN` are removed when the file is read. Those are exactly the cached rows a frame writes when no new sample arrived (write path 2 of §5.2) and any un-indexed legacy samples: rows whose position is a repeat of the previous row's and whose time, by design since the cached row inherits the previous row's τ , is also a repeat. They carry no new information, and dropping them removes the only rows on that path whose time is estimated. The count is printed and kept in `df.attrs["n_unindexed_dropped"]`. On the USB path every row has `RZ2Idx = NaN`, which is why the flag is off for that profile; on an export older than the `RZ2Idx` column the notebook says so and keeps every row.

Epoch filter. `move_epochs` is applied explicitly here rather than trusting the CSV to already be restricted: a row is kept when its `Epoch` matches any listed code. A text `Epoch` column (from an older export) is mapped to numeric codes through `_EPOCH_NAME_TO_CODE` first, and unrecognised names are reported rather than silently dropped. Which codes are loaded differs by cell *by design*: the section 1.1 cell loads (6,7,8) so the target hold is available for the hold flags, while the trajectory-EDA cell and the feature table load (6,7). Because the summary window of §6.7 is now bounded by target entry rather than by the end of the loaded rows, the same trial yields the same summaries under both choices.

Screen geometry. `ScreenGeometry.resolve` takes the hit-test geometry (centre and target diameters, centre-to-target distance and its scale) from the first of these that is available:

1. `orgParams` inside the session's own `params_<runTag>.mat` (`centerRad`, `targetRad`, `centerToTargetDist`), which is the real source of truth for the screen the task rendered that day;
2. a `geometry_spec*.json` or `geometry_spec*.csv` sitting next to the session's `trial_data*.csv`, kept as a manual override for sessions that predate the params export (accepted layouts: a JSON object, a two-column `key,value` CSV, or a one-row CSV with the keys as headers);
3. the module constants (`CENTER_DIAMETER_PX = 200`, `TARGET_DIAMETER_PX = 180`, `CENTER_TO_TARGET_DIST_PX = 320`, `TARGET_DIST_SCALE = 1.27`).

Any key missing from a file keeps its default, and the printed line names the source and lists what fell back. The resulting target radius is what `hold_unstable` is compared against (§6.8).

6.2 Pipeline order

For one trial, with samples (t_k, x_k, y_k) , $k = 1, \dots, n$:

Pre-processing (`TrajectoryProcessor.process`). Times are converted to seconds and sorted with a stable sort; duplicate timestamps are removed with `np.unique(t, return_index=True)`, keeping the *first* occurrence, and counted in `n_duplicate_ts`. A trial is flagged *under-resolved* if $n < \text{min_move_samples} = 5$ (or, if `min\_move\_dur_s` is set, if its duration is below it); kinematics are then returned as `NaN` but the filtered trace is still produced, so the trial still appears in the figures with its flag.

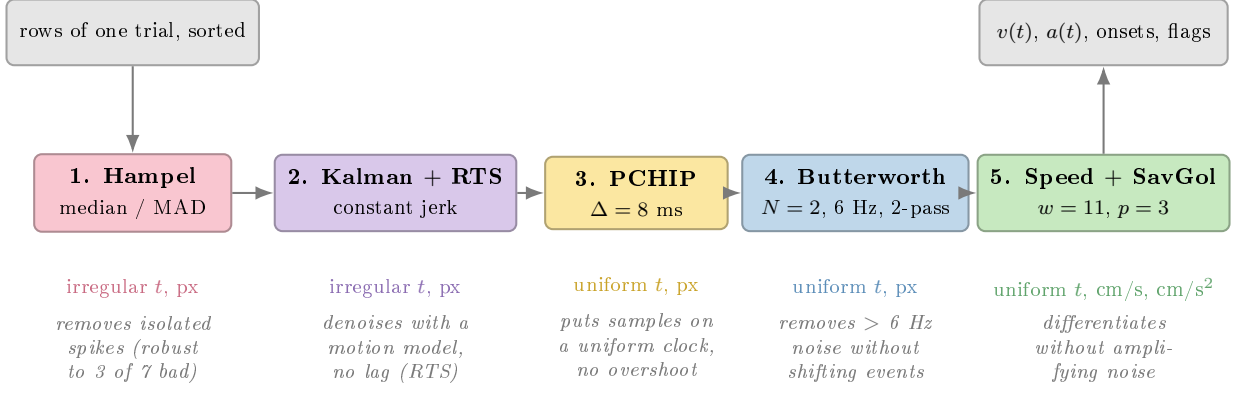


Figure 15: Offline pipeline in `TrajectoryProcessor.process`: each stage, what it is for, and the time base it works in. Stages 1–2 operate on the irregular timestamps; stage 3 creates the uniform grid the last two stages require.

6.3 Stage 1: Hampel screen

Definition. For sample i and half-window h , let $W_i = \{x_j : \max(1, i - h) \leq j \leq \min(n, i + h)\}$ (truncated at the edges). Then

$$m_i = \text{median}(W_i), \quad (19)$$

$$\text{MAD}_i = \kappa \cdot \text{median} \{|x_j - m_i| : x_j \in W_i\}, \quad \kappa = 1.4826, \quad (20)$$

$$\hat{x}_i = \begin{cases} m_i & \text{if } \text{MAD}_i > 0 \text{ and } |x_i - m_i| > n_\sigma \text{MAD}_i, \\ x_i & \text{otherwise,} \end{cases} \quad n_\sigma = 3. \quad (21)$$

Why $\kappa = 1.4826$. If $X \sim \mathcal{N}(\mu, \sigma^2)$ then $\text{median}|X - \mu| = \sigma \Phi^{-1}(0.75) \approx 0.6745\sigma$, so $\kappa = 1/\Phi^{-1}(0.75) = 1.4826$ makes MAD_i a consistent estimator of σ under Gaussian noise, and $n_\sigma \text{MAD}_i$ reads as a 3σ rule.

Why first, and why robust. Median and MAD have breakdown point 50%: up to $\lfloor(2h + 1)/2\rfloor = 3$ gross errors in a 7-sample window leave m_i finite. A single spike that reached the Kalman filter would be interpreted as a real (very large) acceleration and would leak into neighbouring smoothed states, so impulsive errors are removed before any model-based stage.

The $\text{MAD} > 0$ guard. In a window where at least 4 of 7 values are identical (a stationary cursor, integer pixel quantisation), $\text{MAD}_i = 0$ and the rule would flag any deviation at all. The guard leaves such windows untouched; this is a design choice, not a defect, but it means the Hampel stage is inert during perfectly still holds.

Implementation. `HampelScreen.detect` loops explicitly over samples and columns and returns four arrays: `cleaned`, the boolean `mask`, and the envelopes `median_env` and `threshold_env` used by the per-trial figure. Complexity is $O(nh \log h)$ per axis, negligible at these sizes.

6.4 Stage 2: constant-jerk Kalman filter with RTS smoother

Continuous-time model. Per axis, let $\mathbf{s}(t) = [p, v, a]^\top$ (position, velocity, acceleration) and model jerk as white noise $w(t)$ with power spectral density q :

$$\dot{\mathbf{s}} = \mathbf{A}\mathbf{s} + \mathbf{G}w, \quad \mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{G} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbb{E}[w(t)w(t')] = q\delta(t - t'). \quad (22)$$

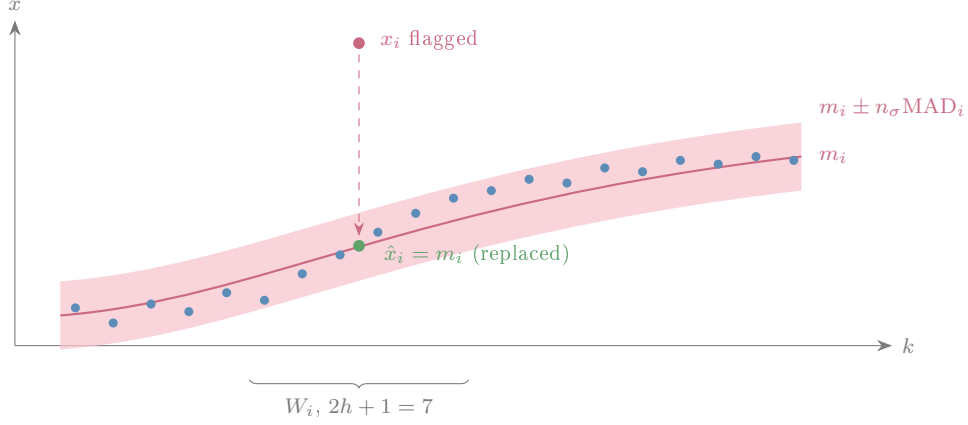


Figure 16: Hampel screen, Eq. (21): the flagged sample is replaced by the local median.

Exact discretization over a step Δt . Because $\mathbf{A}$ is nilpotent ($\mathbf{A}^3 = 0$), the transition matrix is a finite sum,

$$\mathbf{F}(\Delta t) = e^{\mathbf{A}\Delta t} = \mathbf{I} + \mathbf{A}\Delta t + \frac{1}{2}\mathbf{A}^2\Delta t^2 = \begin{bmatrix} 1 & \Delta t & \Delta t^2/2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}, \quad (23)$$

and the process-noise covariance is the integral

$$\mathbf{Q}(\Delta t) = \int_0^{\Delta t} \mathbf{F}(\varsigma) \mathbf{G} \mathbf{q} \mathbf{G}^\top \mathbf{F}(\varsigma)^\top d\varsigma = \mathbf{q} \begin{bmatrix} \Delta t^5/20 & \Delta t^4/8 & \Delta t^3/6 \\ \Delta t^4/8 & \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^3/6 & \Delta t^2/2 & \Delta t \end{bmatrix}, \quad (24)$$

since $\mathbf{F}(\varsigma)\mathbf{G} = [\varsigma^2/2, \varsigma, 1]^\top$ and entry (i, j) integrates the product of the corresponding components. Both matrices are recomputed at every step with the trial's own $\Delta t_k = t_k - t_{k-1}$; this is what makes the filter exact on irregular timestamps, and it is the property that a fixed-frequency filter cannot offer.

Measurement. Only position is observed:

$$z_k = \mathbf{H} \mathbf{s}_k + \nu_k, \quad \mathbf{H} = [1 \ 0 \ 0], \quad \nu_k \sim \mathcal{N}(0, R), \quad R = \sigma_{\text{meas}}^2 = 2^2 \text{ px}^2. \quad (25)$$

Initialization. With $\Delta t_0 = t_1 - t_0$ and $\sigma_{a,0} = 20 \sigma_{\text{jerk}} \text{median}_k(\Delta t_k)$,

$$\hat{\mathbf{s}}_0^- = [z_0, (z_1 - z_0)/\Delta t_0, 0]^\top, \quad \mathbf{P}_0^- = \text{diag}(R, 2R/\Delta t_0^2, \sigma_{a,0}^2). \quad (26)$$

The velocity variance $2R/\Delta t_0^2$ is the variance of a finite difference of two independent measurements; the factor 20 in $\sigma_{a,0}$ is a heuristic inflation so the acceleration prior is uninformative.

Forward recursion (predict / update), $k = 1, \dots, n-1$.

$$\hat{\mathbf{s}}_k^- = \mathbf{F}_k \hat{\mathbf{s}}_{k-1}, \quad \mathbf{P}_k^- = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}_k, \quad (27)$$

$$y_k = z_k - \hat{p}_k^-, \quad S_k = [\mathbf{P}_k^-]_{11} + R, \quad (28)$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}^\top / S_k = [\mathbf{P}_k^-]_{:,1} / S_k, \quad (29)$$

$$\hat{\mathbf{s}}_k = \hat{\mathbf{s}}_k^- + \mathbf{K}_k y_k, \quad \mathbf{P}_k = \mathbf{P}_k^- - \mathbf{K}_k [\mathbf{P}_k^-]_{1,:}, \quad (30)$$

followed by symmetrization $\mathbf{P}_k \leftarrow (\mathbf{P}_k + \mathbf{P}_k^\top)/2$ for numerical hygiene. The update is skipped (the prediction is carried forward and the counter `n_gated` incremented) when the innovation fails the χ^2 gate $y_k^2 > \gamma^2 S_k$; with the default $\gamma = \infty$ the gate never fires.

Backward recursion (Rauch–Tung–Striebel). Starting from $\hat{s}_{n-1}^s = \hat{s}_{n-1}$, for $k = n - 2, \dots, 0$:

$$C_k = P_k F_{k+1}^\top (P_{k+1}^-)^{-1}, \quad \hat{s}_k^s = \hat{s}_k + C_k (\hat{s}_{k+1}^s - \hat{s}_{k+1}^-). \quad (31)$$

Only the state is smoothed (the smoothed covariance is not needed downstream). The stage output is the smoothed position $\hat{p}_k^s$; the smoothed velocity and acceleration are discarded, because the kinematics are recomputed after resampling and low-pass filtering. Figure 17 shows the arrays the implementation keeps so that Eq. (31) can run backward, and Figure 18 shows why the backward pass matters.

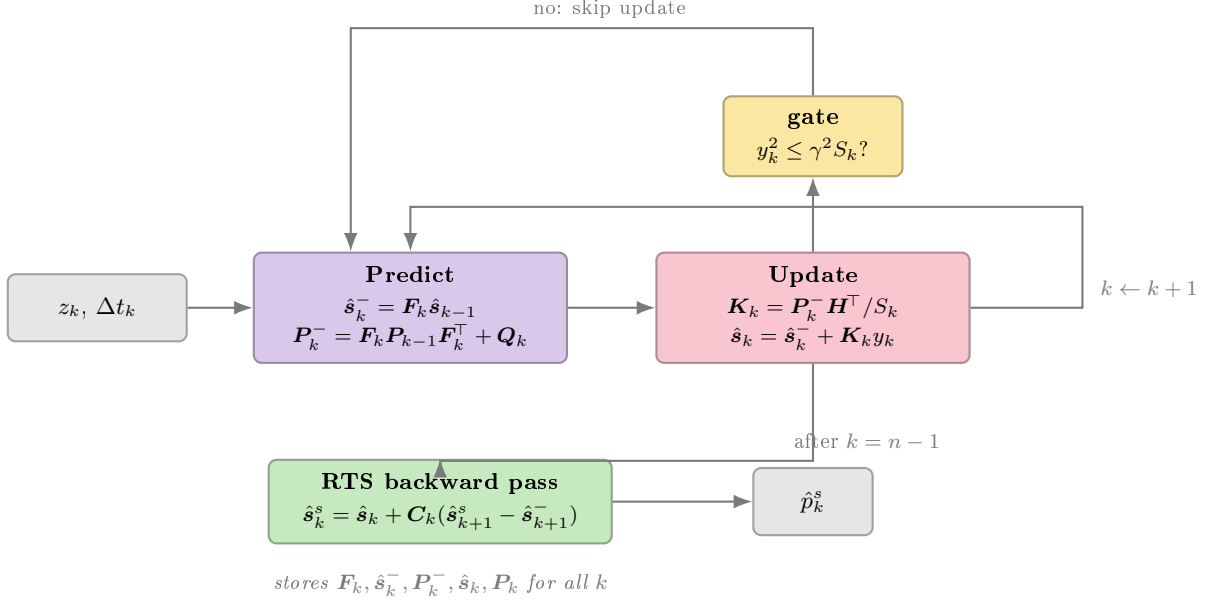


Figure 17: Structure of `KalmanTrajectorySmoother.__call__`. The arrays `xp`, `pp`, `xf`, `pf`, `fk` hold the predicted and filtered quantities so Eq. (31) can run backward.

What the parameters actually do. The filter has one free ratio, q/R . A useful order-of-magnitude rule for a ν -th order integrated-white-noise model observed in position is that the steady-state estimator behaves like a low-pass with corner $\omega_0 \sim (q/R)^{1/(2\nu)}$; here $\nu = 3$, so

$$\omega_0 \sim \left(\frac{q}{R}\right)^{1/6} = \left(\frac{(10^5)^2}{2^2}\right)^{1/6} \approx 37 \text{ rad/s} \approx 5.9 \text{ Hz}. \quad (32)$$

With the defaults ($\sigma_{\text{jerk}} = 10^5 \text{ px/s}^3$, $\sigma_{\text{meas}} = 2 \text{ px}$) the Kalman stage therefore smooths at roughly the same scale as the 6 Hz Butterworth that follows, which is consistent. Raising σ_{jerk} makes the stage closer to a pass-through; lowering it increases smoothing and, because the RTS pass is non-causal, does so without lag. This scaling is a heuristic, not an exact bandwidth; it is offered to give the two parameters an interpretable scale.

6.5 Stage 3: PCHIP resampling to a uniform grid

Grid. `_build_grid` produces $t_g^{(m)} = t_0 + m\Delta$, $m = 0, 1, \dots$, up to t_{n-1} , with $\Delta = \text{grid_dt_s} = 0.008 \text{ s}$. The tail is handled so the last grid point equals t_{n-1} : if the leftover $t_{n-1} - t_g^{(M)}$ exceeds $\Delta/4$ the endpoint is appended, otherwise the last grid point is moved onto it. The final interval is therefore not exactly Δ ; the kinematics use the actual `diff` of the grid, so this introduces no error. If the whole trial is shorter than Δ the grid is just $[t_0, t_{n-1}]$.

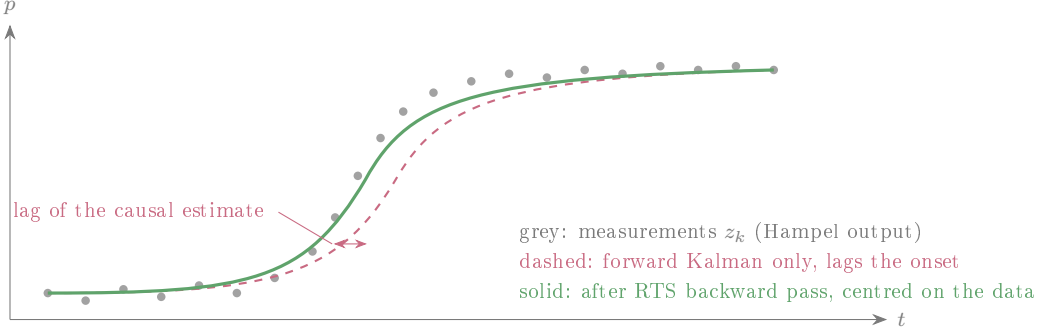


Figure 18: Why the backward pass matters: a causal filter can only react after the change has happened, so its estimate of a movement onset is late by roughly its time constant. The RTS pass revisits every estimate with the whole trial in hand and removes that delay, which is what makes onset and peak times usable offline.

Interpolant. On each interval $[t_i, t_{i+1}]$ PCHIP builds the cubic Hermite polynomial

$$P(t) = \hat{p}_i^s h_{00}(u) + \delta_i d_i h_{10}(u) + \hat{p}_{i+1}^s h_{01}(u) + \delta_i d_{i+1} h_{11}(u), \quad u = \frac{t - t_i}{\delta_i}, \quad \delta_i = t_{i+1} - t_i, \quad (33)$$

with the standard Hermite basis $h_{00}, h_{10}, h_{01}, h_{11}$ and endpoint slopes d_i chosen by the Fritsch–Carlson rule: if the secants $\sigma_{i-1} = (\hat{p}_i^s - \hat{p}_{i-1}^s)/\delta_{i-1}$ and σ_i have opposite signs or one is zero, then $d_i = 0$; otherwise d_i is the weighted harmonic mean

$$d_i = \frac{w_1 + w_2}{w_1/\sigma_{i-1} + w_2/\sigma_i}, \quad w_1 = 2\delta_i + \delta_{i-1}, \quad w_2 = \delta_i + 2\delta_{i-1}. \quad (34)$$

The result is C^1 , preserves monotonicity of the data between samples, and cannot overshoot at a sharp movement onset, unlike a natural cubic spline. The second derivative is discontinuous at the knots, which is acceptable because acceleration is never taken from this interpolant.

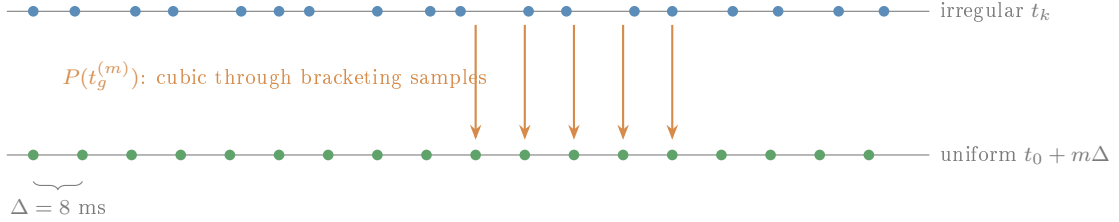


Figure 19: Resampling from the raw irregular clock onto the 125 Hz grid.

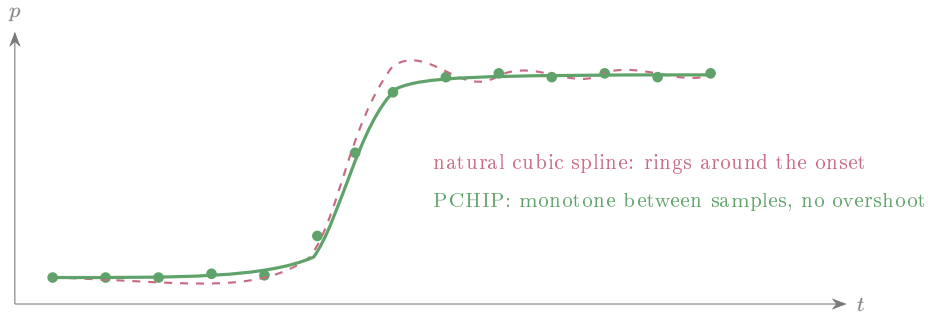


Figure 20: Why PCHIP rather than a smoothing spline: at a reaching onset the data go from still to moving within one or two samples, and a spline invents oscillations there that a differentiator would later read as acceleration.

Rate reduction. Raw `rz2adc` data arrive at ≈ 939 Hz, so the 125 Hz grid is a $\sim 7.5\times$ decimation. No anti-alias filter is applied between stages 2 and 3: the Kalman stage acts as one (see Eq. (32)) and any residual content between 6 Hz and 62.5 Hz is removed by stage 4. Content above the new Nyquist frequency (62.5 Hz) would alias, but joystick signals that survive stage 2 have no meaningful power there. If a much larger σ_{jerk} were chosen (weaker Kalman smoothing), this assumption should be re-checked.

6.6 Stage 4: zero-phase Butterworth low-pass

Analog prototype. A second-order Butterworth low-pass with cutoff ω_c has

$$|H_a(j\omega)|^2 = \frac{1}{1 + (\omega/\omega_c)^4}, \quad H_a(s) = \frac{1}{\bar{s}^2 + \sqrt{2}\bar{s} + 1}, \quad \bar{s} = s/\omega_c. \quad (35)$$

Bilinear transform with prewarping. Substituting $\bar{s} = \frac{1}{K} \frac{1-z^{-1}}{1+z^{-1}}$ with $K = \tan(\pi f_c/f_s)$ maps the analog cutoff exactly onto the digital f_c and gives

$$H(z) = \frac{K^2 (1 + 2z^{-1} + z^{-2})}{(1 + \sqrt{2}K + K^2) + 2(K^2 - 1)z^{-1} + (1 - \sqrt{2}K + K^2)z^{-2}}. \quad (36)$$

Normalizing by $a_0 = 1 + \sqrt{2}K + K^2$ yields exactly the arrays in `ButterworthLowpass._design`:

```
k = np.tan(np.pi * self.cutoff_hz / fs)
norm = 1.0 / (1.0 + np.sqrt(2.0) * k + k ** 2)
b = np.array([k ** 2, 2.0 * k ** 2, k ** 2]) * norm
a = np.array([1.0, 2.0 * (k ** 2 - 1.0) * norm,
              (1.0 - np.sqrt(2.0) * k + k ** 2) * norm])
```

With $f_c = 6$ Hz and $f_s = 125$ Hz, $K = \tan(0.1508) \approx 0.1519$.

Forward–backward application. The filter is run forward, the output is reversed, run again, and reversed back. The effective response is

$$H_{\text{eff}}(e^{j\omega}) = H(e^{j\omega}) H(e^{-j\omega}) = |H(e^{j\omega})|^2, \quad \angle H_{\text{eff}} = 0, \quad (37)$$

so the magnitude is squared (stop-band attenuation doubles in dB) and the phase is identically zero. The price is a shift of the -3 dB point: solving $|H|^4 = 1/2$ gives $(\omega/\omega_c)^4 = \sqrt{2} - 1$, i.e.

$$f_{-3\text{dB}}^{\text{eff}} = (\sqrt{2} - 1)^{1/4} f_c \approx 0.802 f_c \approx 4.8 \text{ Hz}. \quad (38)$$

For p passes of an order- N Butterworth the general ratio is $(2^{1/p} - 1)^{1/(2N)}$. The notebook computes the digital value on the prewarped axis rather than quoting the analog one, so the figure titles stay correct if either parameter changes:

```
def effective_cutoff_hz(cutoff_hz=CUTOFF_HZ, fs_hz=1.0 / GRID_DT_S, order=2, n_passes=2):
    ratio = (2.0 ** (1.0 / n_passes) - 1.0) ** (1.0 / (2.0 * order))
    return float(fs_hz / np.pi * np.arctan(ratio * np.tan(np.pi * cutoff_hz / fs_hz)))
EFFECTIVE_CUTOFF_HZ = effective_cutoff_hz() # 4.82 Hz at fc = 6, fs = 125
```

This is the number to keep in mind when reading “6 Hz”: the two-pass filter is half-power at about 4.8 Hz and reaches -6 dB at 6 Hz.

Edge handling. Two devices suppress start-up transients, both mirroring `scipy.signal.filtfilt` defaults:

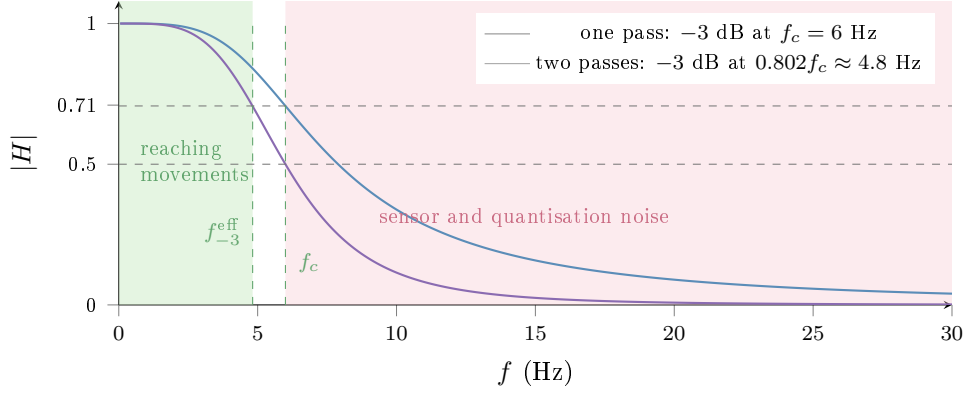


Figure 21: Magnitude response of the analog prototype (one pass) and of the forward–backward application (two passes). The bilinear transform preserves these curves up to frequency warping, which the prewarped K pins exactly at f_c .

1. *Odd reflection padding* of $n_f = 6$ samples at each end, $\tilde{x}_{-j} = 2x_0 - x_j$ and $\tilde{x}_{n-1+j} = 2x_{n-1} - x_{n-1-j}$, $j = 1..n_f$; the padded region is discarded after filtering (`ext[nf:-nf]`). Here $n_f = 3 \cdot \max(\text{len}(a), \text{len}(b)) - 3 = 6$.
2. *Steady-state initial conditions*. For the transposed direct-form II realization with state ζ , the state that reproduces a constant input x as a constant output satisfies $(\mathbf{I} - \mathbf{A}_c)\zeta = \mathbf{b}'$ with companion matrix $\mathbf{A}_c = \begin{bmatrix} -a_1 & 1 \\ -a_2 & 0 \end{bmatrix}$ and $\mathbf{b}' = [b_1 - a_1b_0, b_2 - a_2b_0]^\top$. `_steady_state` solves this once and scales the result by the first padded sample (`zi * ext[0]`).

The filter is skipped (`applied = False`) when $n \leq n_f$ or when $f_c \geq f_s/2$, and the per-trial figure title says which of the two happened.

Why here and why zero-phase. Voluntary reaching movements carry essentially no power above a few Hz; what remains above ~ 6 Hz on the 125 Hz grid is quantisation and sensor noise. A causal filter would delay every kinematic event by its group delay (tens of ms at these settings), which would bias movement-onset and peak-speed times; zero phase removes the noise without moving any feature.

6.7 Stage 5: speed, Savitzky–Golay acceleration, summary window

Speed. On the grid, with actual steps $g_k = t_g^{(k+1)} - t_g^{(k)}$ (all $= \Delta$ except possibly the last),

$$v_k = \frac{\sqrt{(x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2}}{g_k}, \quad t_k^{(v)} = t_g^{(k)} + \frac{g_k}{2}, \quad v_k^{\text{cm}} = c v_k. \quad (39)$$

This is a central-difference estimate located at the segment midpoint (`np.hypot(np.diff(...))` divided by `np.diff(t)`). Because $v_k \geq 0$ by construction it is speed, not signed velocity.

Tangential acceleration via Savitzky–Golay. Rather than differencing v_k again, in a sliding window of $w = 11$ samples (≈ 88 ms at 125 Hz) a polynomial of degree $p = 3$ is fitted by least squares and differentiated analytically at the window centre:

$$\hat{v}(u) = \sum_{j=0}^p \beta_j u^j, \quad \boldsymbol{\beta} = (\mathbf{V}^\top \mathbf{V})^{-1} \mathbf{V}^\top \mathbf{v}_{k-h:k+h}, \quad a_k = \frac{|\beta_1|}{\Delta}, \quad (40)$$

where $\mathbf{V}$ is the $w \times (p+1)$ Vandermonde matrix on $u = -h, \dots, h$, $h = (w-1)/2$. Because $\mathbf{V}$ is fixed, $(\mathbf{V}^\top \mathbf{V})^{-1} \mathbf{V}^\top$ is a fixed $(p+1) \times w$ matrix and a_k is a convolution of $\mathbf{v}$ with its second

row (the derivative filter). The absolute value is taken, so a_k is the magnitude of the tangential acceleration $|d\mathbf{v}|/dt$, in cm/s^2 . The step passed to `savgol_filter` is $\text{median}(g_k)$, not Δ , so a grid whose last interval was shortened is handled correctly.

Why not a second finite difference. Differentiation multiplies the noise spectrum by ω^2 (in power); a bare difference of v_k would let single-sample jitter dominate $\max_k a_k$. The least-squares fit acts as a low-pass before the derivative is taken, and the 99th percentile (`ACCEL_PEAK_PERCENTILE`) is used as the “peak” so that one residual spike does not set the value.

Window fallbacks (`_savgol_accel`). $w \leftarrow \min(w, n \text{ or } n - 1)$ forced odd and ≥ 5 ; $p \leftarrow \min(p, w - 1)$. For $n < 5$ the code uses `np.gradient` instead; $n = 1$ returns 0 and $n = 0$ returns an empty array.

Summary window: takeoff to target entry. Every per-trial summary is taken over the same samples. Let $\mathcal{S}$ be the speed samples whose midpoint time falls at or before the last `MOVEMENT` sample (target entry), taken from the `Epoch` column; if the trial has no `MOVEMENT` epoch (an early exit) the search falls back to all samples.

Which peak the walk-back starts from. Inside $\mathcal{S}$ the local maxima of v are found with `scipy.signal.find_peaks` under a prominence threshold of `MULTI_PEAK_PROMINENCE_FRAC` = 0.15 times the largest speed in $\mathcal{S}$, and their count is recorded as `n_speed_peaks` (at least 1). With `TAKEOFF_PEAK_CHOICE` = “first”, the default, k^* is the *first* of those peaks; with any other value it is the global $\arg \max_{k \in \mathcal{S}} v_k$, which is what the code did before. The distinction only matters on a trial whose speed profile has more than one submovement: walking back from the global maximum, which may belong to the second submovement, would place the takeoff after the first one had already started, and the movement onset would be late by the whole duration of the first. The first prominent peak is the one that belongs to the initial reach, and dating the onset is what the takeoff exists for.

The walk-back. With k^* fixed,

$$\theta = \rho v_{k^*}, \quad k_{\text{takeoff}} = \min\{k \leq k^* : v_j \geq \theta \ \forall j \in [k, k^*]\}, \quad (41)$$

that is, walk backward from k^* while the speed stays above a fraction ρ of it; `move_takeoff_ms` is $t_{k_{\text{takeoff}}}^{(v)}$. The fraction is not a bare constant: `TAKEOFF_SPEED_FRAC_OPTIONS` offers “5%” = 0.05 and “10%” = 0.10, and `TAKEOFF_SPEED_FRAC_CHOICE` (default “5%”) selects the one that `MOVE_SPEED_FRAC` takes. Both are set with `globals().get(...)`, so a cell run earlier in the notebook can override either without editing the pipeline.

The window. The summary window is $\mathcal{M} = \{k \in \mathcal{S} : k \geq k_{\text{takeoff}}\}$, and *every* statistic uses it:

$$\begin{aligned} v_{\text{peak}} &= \max_{k \in \mathcal{M}} v_k, & v_{\text{mean}} &= \text{mean}_{k \in \mathcal{M}} v_k, & v_{\text{med}} &= \text{median}_{k \in \mathcal{M}} v_k, \\ a_{\text{peak}} &= \text{P}_{99}(\{a_k\}_{k \in \mathcal{M}}), & a_{\text{mean}} &= \text{mean}_{k \in \mathcal{M}} a_k, & a_{\text{med}} &= \text{median}_{k \in \mathcal{M}} a_k. \end{aligned} \quad (42)$$

Note that v_{peak} is the maximum over $\mathcal{M}$, not v_{k^*} : on a two-submovement trial the reported peak can belong to the second submovement while the takeoff was walked back from the first, and `peak_vel_time_ms` records where that maximum actually is.

Restricting the search to target entry is what stops a jitter during the target hold from being taken as the reach peak, and using one mask for all six statistics is what makes them comparable with each other. It is also what makes the same trial give the same summaries in the section 1.1 cell (which loads epochs 6,7,8), in the trajectory-EDA variant, and in the feature table (which load 6,7): the window no longer depends on which rows happen to be loaded. `window_start_ms` and `window_end_ms` record the window actually used, and their difference is exported as `takeoff_to_target_ms` and, in seconds, as `TakeoffToTargetTime_s`.

Takeoff lead. $\text{takeoff_lead_ms} = \text{move_onset_ms} - \text{move_takeoff_ms}$ is how long the hand was already moving when it crossed the centre circle, where move_onset_ms is the first `MOVEMENT` timestamp, that is, the row the task used for `t.leaveCenter`. Both times come from rows of the same trajectory, so the transport latency cancels in the difference. The derived column $\text{TakeoffTime_s} = \text{DecisionTime_s} - \text{takeoff_lead_ms}/1000$ places the takeoff on the go signal and the clock of `DecisionTime_s`.

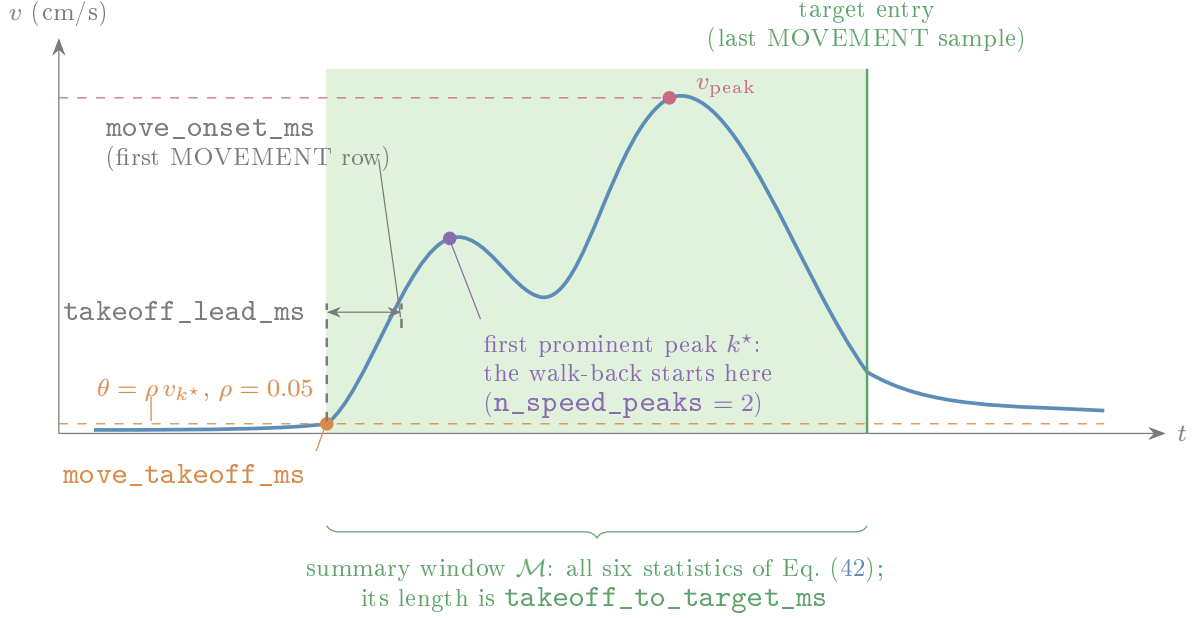


Figure 22: The summary window on a trial with two submovements. Local peaks are found inside the search region (up to target entry); the walk-back to $\theta = 5\%$ starts from the *first* prominent one, so the takeoff dates the initial reach rather than the second submovement. Peak, mean and median of speed and acceleration are then taken over the whole shaded interval, which is why v_{peak} can belong to a later peak than k^* .

6.8 Data-quality flags

Three flags are computed, two of them from raw rows in `TrajectoryDataset.trials` and one in `TrialFigure.build`:

$$\text{under_resolved} \iff n < 5, \quad (43)$$

$$\text{hold_unstable} \iff e_{\text{hold}} > r_{\text{target}}, \quad e_{\text{hold}} = \max_{k \in \text{hold}} \sqrt{(x_k - x_{h_0})^2 + (y_k - y_{h_0})^2}, \quad (44)$$

$$\text{hold_incomplete} \iff d_{\text{hold}} < 0.8 \cdot \text{median}_{\text{session}}(d_{\text{hold}}), \quad d_{\text{hold}} = \tau_{\text{hold},\text{end}} - \tau_{\text{hold},\text{start}}, \quad (45)$$

where h_0 is the first sample of the `TARGET_HOLD` epoch and r_{target} is the target radius resolved in §6.1. A raw hold speed $\max_k \|\Delta(x, y)\|/\Delta t \cdot c$ (`hold_max_speed_cm`) is also reported. The flags annotate; they never alter the signal, and they appear in the figure title and as columns of the feature table. The title carries one further tag that is not a flag: `[multi-peak xN]` when `n_speed_peaks` > 1 , so a trial whose takeoff was walked back from the first of several peaks says so on its own figure.

What the per-trial figure run reports. `run()` draws one figure per trial and prints four things about the set as a whole.

- *Early exits are not drawn.* Trials whose `ErrorType` equals `EARLY_EXIT_CODE = 1` are collected from the trial table before anything is processed and skipped, with the count printed. No target

was reached, so there is no takeoff-to-target window to show, and the axis limits are computed with those trials excluded (`global_limits(..., exclude_keys)`) so that one abandoned trial cannot set the scale for all the others.

- *The time axis is capped.* `MAX_TRAJ_TIME_MS = 4000` ms is now a hard upper limit rather than a maximum taken with the data: the shared axis ends at 4 s whether or not any trial reaches it, which is what makes two sessions' figures comparable by eye.
- *Trials past the cap are named, not hidden.* `global_limits` returns the over-cap trials as its third element and `run()` prints each one by Block, TrialNumInBlock, Attempt and its actual duration, or states that none exceeded the cap. A clipped trace is still drawn; the point is that clipping is never silent.
- *Multi-peak trials are listed.* Every trial with `n_speed_peaks > 1` is printed the same way, with the `TAKEOFF_PEAK_CHOICE` in force, since that is the population in which the choice of k^* changes the answer.

The per-trial statistics box on each figure reports **Takeoff-to-target time** alongside the decision, execution and total times, so the trajectory's own interval sits next to the three the task CSV supplies.

6.9 Offline clock check

`RZ2Idx` makes the online time base auditable. When the full trajectory export carries the column and the session is detected as `rz2adc`, `OfflineClockFit` fits

$$\tau_i = b + a n_i \quad (46)$$

by ordinary least squares over the indexed rows (sorted by index, using only rows with finite n_i , finite τ_i and $\tau_i > 0$), and reports:

- the implied write rate $1000/a$ Hz and its deviation in parts per million from the seed 24414.0625/26 Hz;
- the residuals of the online `Time_ms` about that line, as RMS, 99th percentile of $|r|$, and $\max |r|$;
- the number of backward steps of `Time_ms` in index order, and the worst one.

A healthy session shows a rate within a few hundred ppm of the seed, small residuals, and no backward steps. This is an independent, whole-session check on a quantity the online estimator produced incrementally from a 600-observation window: the two use the same data but not the same estimator, so agreement is evidence and disagreement localises the problem. The analyses still read `Time_ms`; the fit is a check on it, not a replacement for it. On a USB or mouse session there are no indexed rows and the report says so.

Where the check runs. `OfflineClockFit` is called from the section 1.1 cell of the single-session notebook, on the full trajectory export of the one session being examined. The multi-session notebook no longer carries it: the per-session table it prints after `build_multi_session_features` is now `TAKEOFF_TO_TARGET_TABLE`, one row per session with `Session`, `InputSource`, `n_trials`, `n_with_takeoff`, and the mean and median of `TakeoffToTargetTime_s`. The consequence is worth stating plainly, since the two tables answer different questions: a folder of sessions is now summarised by a kinematic quantity and no longer by a per-session clock report, so the time base of a session in a multi-session sweep is checked by opening it in the single-session notebook. The check itself is unchanged and still runs on the full export, which is the file with the pre-movement rows; the movement-only export carries `RZ2Idx` as well, but only over the response windows, so a slope fitted on it would be blind to the inter-trial gaps where a drift accumulates unobserved.

Sessions the loader excludes. The notebook’s session loader carries a dictionary `INVALID_SESSIONS` and a switch `EXCLUDE_INVALID_SESSIONS` (default `True`). Two sessions are listed, both recorded with the pre-clock-map fixed anchor, whose RZ2 stamps drifted from `GetSecs` at ≈ 13.7 ms/s: `sessROM_31-Aug-2026_14-20` and `sessPX-309`. Their decision times are recoverable by adding the fitted divergence, but from the trial where the effective hold reached zero the protocol was no longer the configured one, so those trials are not comparable with anything. The exclusion is a property of the current loader, and the reason string is printed when a session is skipped.

7 Downstream analyses on the kinematics

The single-session notebook runs the pipeline of §6 unchanged (its trajectory-EDA cell differs in figure styling and in the epochs it loads) and then consumes its output in eight analyses. This section describes the ones that read the trajectory itself, and says where the timing columns of `trial_data_*.csv` enter the others, because those columns are what §2.3 and §2.5 correct.

7.1 Per-trial feature vector

`build_trial_features` runs `TrajectoryProcessor.process` on every trial and joins the result to `trial_data_*.csv` on (Block, TrialNumInBlock, Attempt). The trial table is first passed through `normalize_trial_schema`, which canonicalises four generations of timing headers onto `DecisionTime_s` (target onset to leave centre), `ExecutionTime_s` (leave centre to reach target) and `TotalTime_s` (their sum). The one signal that separates the generations is whether `ExecutionTime_s` is present: if it is, a `ReactionTime_s` column is the total; if it is not, `ReactionTime_s` is the execution time. `ReactionTime_s` is dropped after being folded in, because keeping both would put two identical columns into the correlation heatmap and the PCA, where an exact duplicate is not a second measurement.

Two path descriptors are added over the filtered grid, between `move_onset_ms` and `move_offset_ms` (the start of the target hold, falling back to the ends of the grid when either is missing):

$$\ell = \sum_k \|\mathbf{g}_{k+1} - \mathbf{g}_k\|_2, \quad s = \frac{\|\mathbf{g}_{\text{end}} - \mathbf{g}_{\text{start}}\|_2}{\ell}, \quad (47)$$

the path length in cm and the *straightness* $s \in (0, 1]$, the ratio of net displacement to distance travelled. A perfectly straight reach has $s = 1$; a mid-flight correction lowers it.

Feature	Source	Depends on the time base
DecisionTime_s, ExecutionTime_s, TotalTime_s	task CSV	yes (§2.3, §2.5)
n_samples, fs_hz	trajectory rows	fs_hz yes
move_takeoff_ms	stage 5, walk-back to θ from the first prominent peak	yes
n_speed_peaks	stage 5, find_peaks on $\mathcal{S}$	no
takeoff_lead_ms	move_onset_ms – move_takeoff_ms	a difference within one trajectory; latency cancels
TakeoffTime_s	DecisionTime_s – takeoff_lead_ms/1000	yes, through DecisionTime_s
takeoff_to_target_ms, TakeoffToTargetTime_s	length of $\mathcal{M}$ (Eq. (41) to target entry)	yes
peak/mean/median_vel_cm	stage 5, window $\mathcal{M}$	yes
peak/mean/median_accel_cm	stage 5, Savitzky–Golay	yes
path_length_cm	filtered grid	no (spatial)
straightness	filtered grid	no (spatial)
hold_dur_ms, hold_max_speed_cm, hold_excursion_px	target-hold epoch	partly
under_resolved	Eq. (43)	no
InputSource	Eq. (18)	no

`build_trial_features` also prints how many trials carry more than one prominent speed peak, which is the population a change-of-mind or a mid-flight correction would live in, together with the `TAKEOFF_PEAK_CHOICE` in force; the per-trial count stays in `n_speed_peaks`.

The returned frame also carries four diagnostics in `df.attrs`: `input_source`, `source_reason`, `n_unindexed_dropped` and `n_duplicate_ts` (plus `n_trials_with_duplicates`), so a figure built from it can state how the session was classified and how many rows the reader removed.

Before any figure, `drop_early_exits` removes trials with `ErrorType = EARLY_EXIT_CODE = 1` (early exit or timeout: the trial ended without a target being chosen). It cross-checks that against `ChosenTarget`, printing a warning when the two disagree (a dropped trial with `ChosenTarget` ≥ 1 , or a kept trial without one) and a confirmation line when they coincide on every trial. The check exists because the two columns are written by different branches of the state machine and nothing else would notice them diverging.

7.2 Changes of mind from the initial heading

This is the analysis that most justifies recording trajectories rather than endpoints. For each trial the cursor’s heading over the first $w = \text{INITIAL_HEADING_MS} = 120$ ms after movement *takeoff* is compared with the target finally chosen.

Let $\mathbf{g}(t)$ be the filtered grid position and $t_0 = \text{move_takeoff_ms}$ (falling back to the first grid sample). The initial displacement and unit heading are

$$\mathbf{d} = \mathbf{g}(t_0 + w) - \mathbf{g}(t_0), \quad \hat{\mathbf{u}} = \frac{\mathbf{d}}{\|\mathbf{d}\|}, \quad \text{valid only if } \|\mathbf{d}\| \geq \text{MIN_HEADING_PX} = 12 \text{ px.} \quad (48)$$

Anchoring on the takeoff rather than on the leave-centre row matters: the hand is already moving when it crosses the centre circle (that is exactly what `takeoff_lead_ms` measures), so a window that started at the crossing would miss the first, most committed part of the reach.

The four targets sit on the cardinal directions. In screen coordinates y grows downward, so

$$\mathbf{e}_{\text{Right}} = (1, 0), \quad \mathbf{e}_{\text{Up}} = (0, -1), \quad \mathbf{e}_{\text{Left}} = (-1, 0), \quad \mathbf{e}_{\text{Down}} = (0, 1). \quad (49)$$

The initial direction is the one with the largest dot product, and its angular error is

$$D^* = \arg \max_D \hat{\mathbf{u}} \cdot \mathbf{e}_D, \quad \vartheta = \arccos(\hat{\mathbf{u}} \cdot \mathbf{e}_{D^*}). \quad (50)$$

A *change of mind* is recorded when D^* differs from the chosen target's direction (`DirectionChosen`), restricted to trials where both are available and `DirectionChosen` is not "None". Straightness is the independent cross-check: a reach that set off toward one target and ended at another must curve.

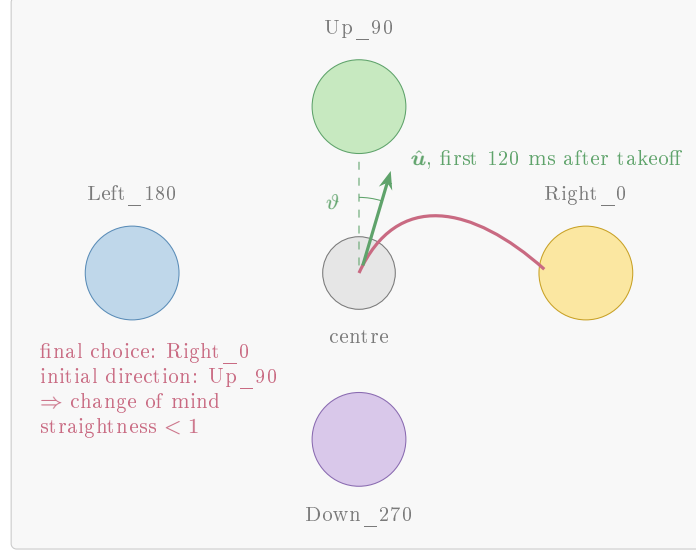


Figure 23: Change of mind from the initial heading. The unit vector over the first 120 ms after takeoff points at Up; the reach ends at Right. Screen y grows downward, which is why Up is encoded as $(0, -1)$ in Eq. (49).

The implementation is `initial_heading`, which returns `(None, dist)` when the cursor has not travelled far enough for the direction to mean anything:

```
t0 = result.move_takeoff_ms if np.isfinite(result.move_takeoff_ms) else gt[0]
mask = (gt >= t0) & (gt <= t0 + window_ms)
if mask.sum() < 2:
    return None, np.nan
seg = gxy[mask]
v = seg[-1] - seg[0]
dist = float(np.hypot(v[0], v[1]))
if dist < min_px:
    return None, dist
return (v / dist), dist
```

Algorithm 2 is the cell as a whole. Line 2 runs the five-stage pipeline for the trial; lines 3–6 are Eq. (48), with the early `continue` on line 6 marking a trial whose heading is unresolved rather than guessing one; lines 7–8 are Eqs. (49)–(50) and the comparison against the logged choice; line 10 is where the result meets the difficulty axis of §7.3.

The 120 ms window is short on purpose: read on the 8 ms grid it spans 15 grid points, and it ends well before the typical execution time. Two properties of the acquisition chain matter here. The window is anchored on `move_takeoff_ms`, which is derived from speed and therefore from the time base; and the heading is a spatial quantity on the filtered grid, so it is insensitive to a constant latency but not to a stretched or non-monotone time axis, which would move the takeoff.

Algorithm 2: build_change_of_mind and change_of_mind_stats.

```
1 foreach trial with epochs {decision, movement} do
2    $r \leftarrow \text{TrajectoryProcessor.process}(\text{trial})$ 
3    $t_0 \leftarrow r.\text{move\_takeoff\_ms}$  (or the first grid sample)
4    $\text{seg} \leftarrow r.\text{grid\_xy}[t_0 \leq t \leq t_0 + 120 \text{ ms}]$ 
5    $\mathbf{d} \leftarrow \text{seg}[\text{end}] - \text{seg}[0]$ 
6   if  $\|\mathbf{d}\| < 12 \text{ px}$  then
7     mark unresolved; continue
8   end
9    $\hat{\mathbf{u}} \leftarrow \mathbf{d}/\|\mathbf{d}\|$ ;  $D^* \leftarrow \arg \max_D \hat{\mathbf{u}} \cdot \mathbf{e}_D$ ;  $\vartheta \leftarrow \arccos(\hat{\mathbf{u}} \cdot \mathbf{e}_{D^*})$ 
10   $\text{changed\_mind} \leftarrow [D^* \neq \text{DirectionChosen}]$ 
11 end
12 Report the rate of changes of mind against distance to the fitted boundary (Spearman  $\rho$ ),
    and compare straightness, TotalTime_s and DecisionTime_s between the two groups
    (Mann–Whitney)
```

7.3 Where the timing columns enter the other analyses

Psychometric functions (1.5). No kinematics and no timing. Each of the $K - 1$ category boundaries gets its own cumulative Gaussian with a lapse term, fitted by binomial maximum likelihood to the ordinal splits $P(\text{chosen} \geq 2)$ and $P(\text{chosen} \geq 3)$; 95% intervals are percentile bootstraps over the per-level binomial counts. Only **ErrorType** matters here, through the lapse term, and only type 3 (hold break after a target was chosen) reaches the fit: it puts a floor and a ceiling on the curve, and fitting without a lapse absorbs those trials into the slope and biases both μ and σ . Type 1 (early exit) never arrives, because **ordinal_choice_table** keeps only **ChosenTarget** ≥ 1 and the source frame is **ANALYSIS_DF** when it exists and **drop_early_exits(FEATURES_DF)** otherwise, so the cell no longer depends on an earlier cell having filtered for it. A reduced stimulus set (three lengths) cannot identify three free parameters per boundary, and the cell reports raw proportions instead of a fit the data cannot support.

Chronometric functions (1.6). Difficulty is the distance from each trial’s bar length to the nearest *fitted* boundary (**boundary_distance**, using the μ values from 1.5 rather than the nominal [5.05, 6.40] deg VA), so it reflects where this subject placed the criterion. **TotalTime_s**, **DecisionTime_s**, **ExecutionTime_s** and **TakeoffToTargetTime_s** are tested separately, by Spearman ρ and by linear regression, because the decision half is where a difficulty effect belongs and a signature in the execution half would indicate a motor confound. The fourth measure is the only one read off the trajectory rather than off the task CSV, and it is the execution half timed from the movement instead of from the geometry: **ExecutionTime_s** starts when the cursor leaves the centre circle, **TakeoffToTargetTime_s** starts at the takeoff, which is earlier by **takeoff_lead_ms**. The two therefore differ by that lead, up to the residual between the task’s own event marks and the trajectory rows they were taken from. This is the analysis most directly damaged by a clock defect: a session-long trend in **DecisionTime_s** would masquerade as anything correlated with trial order.

Sequential effects (1.7). Post-error slowing uses the same timing columns; direction perseveration uses **DirectionChosen** only. Because **CenterOutTask.m** carries **prevTrialCorrect** across block boundaries, the previous outcome is recomputed within each block and the first trial of every block is dropped; the agreement rate against the logged column is printed as a check on the CSV.

Error taxonomy and foil events (1.8). `ErrorType` separates type 2 (a target was chosen and it was the wrong one, the only category error) from types 1 and 3 (motor or attention failures where no category was committed to). Perceptual errors should concentrate near the boundary while motor errors should be flat against difficulty, and pooling them is what makes a lapse look like a discrimination failure. `foil_events_*.csv` is read here for the per-entry record of every distractor touch during forgiving pre-training, which gives a graded measure of uncertainty that a one-row-per-trial file cannot express.

PCA, K-means, decoding (1.3, 1.4, 1.10). These consume the feature vector above; `takeoff_to_target_ms` is in both the `PCA_FEATURES` list and the `DECODE_PREFERRED` list, next to `move_takeoff_ms`. `BarSizeVA_deg` is held out when the target is `StimulusGroup`, since the category is a deterministic function of the bar length and leaving it in would score a lookup table. A permutation test sets the chance level for the actual sample size and class balance rather than assuming $1/n_{\text{classes}}$, and the adjusted Rand index of the K-means solution is printed alongside for comparison. K-means chooses k by silhouette; when that choice is not 2 and `SHOW_K2_OVERLAY` is `True` (the default), the cell additionally fits $k = 2$ and cross-tabulates it against `IsCorrect`. The fixed- k view exists because the silhouette answers “how many compact clusters are in this feature space” and not “do the two outcome labels separate”, and only the second question is the one the panel is read for.

Figure colours. `CATEGORY_COLORS` is derived from the rig’s own category colours rather than from a plotting palette. `ColorCategoryMap.m` paints category 1 orange, 2 green and 3 blue, and `ConfigOrgParams.m` exposes the same three as `color3CatShort/Mid/Long`; the single-session notebook reads them from this session’s `params_<runTag>.mat` (`SESSION_PARAMS.color3cat_hex`) when the snapshot is present and falls back to the literals `FFA500`, `00FF00`, `0000FF` otherwise. Each is then passed through `_pastelize`, which converts to HLS, keeps the hue and pins lightness and saturation to $(0.72, 0.60)$. Keeping the hue is what makes a figure agree with what the screen painted; pinning the other two is what keeps three colours chosen for a dark booth display from being read as ink.

8 Parameter reference: what to modify, where, and what it does

8.1 Online chain (MATLAB)

Parameter	Default	Location	Effect and guidance
<code>rz2SampleRateHz</code> (f_s^{seed})	939.0024	<code>ConfigOrgParams.m</code> , read in <code>SetupRZ2Joystick.m</code>	Seed of the clock map slope and centre of its clamp (Eq. (2)). Should equal $24414.0625/\text{downsample}$; confirm with <code>getSamplingRates</code> . A wrong seed costs a warm-up, not a session, but a seed outside $\pm 1\%$ of reality makes the clamp bind and trips the monitor.
<code>rz2ClockWindow</code> (L)	600	<code>ConfigOrgParams.m</code>	Observations (one per accepted drain) in the sliding OLS window (≈ 16 obs/s on this link, so ≈ 40 s). Slope window only.

Parameter	Default	Location	Effect and guidance
<code>rz2ClockMinSpanSec</code>	30 s	<code>ConfigOrgParams.m</code>	Index span the slope window must cover before OLS replaces the seed (Eq. (2)). Below it the seed is the better estimate.
<code>rz2ClockMaxRateDev</code> (δ)	0.01	<code>ConfigOrgParams.m</code>	Fractional clamp on the slope around the seed. The seed is rig-confirmed to $\sim 0.04\%$, so a fit that wants to leave it by more than 1 % is noise or a corrupt index stream.
<code>rz2ClockOffsetWindow</code>	100	<code>ConfigOrgParams.m</code>	Most recent observations the intercept quantile is taken over (Eq. (5)). Must slide: a window that never fills ratchets.
<code>rz2ClockOffset-Quantile</code>	0.10	<code>ConfigOrgParams.m</code>	Residual quantile the intercept is anchored on; 0 would be the strict minimum, which one early packet can drag.
<code>rz2ClockMaxSlewSec</code>	2 ms	<code>ConfigOrgParams.m</code>	Largest move one refit may apply to $\hat{t}$ at the newest index ($\times 5$ during the first 100 fits). Keeps the output monotone by design.
<code>rz2SkewWarnSec,</code> <code>rz2SkewAbortSec</code>	0.05, 0.20 s	<code>ConfigOrgParams.m</code> $\rightarrow$ <code>ClockSkewMonitor.m</code>	Warn threshold on the instantaneous $ \zeta_k $; abort threshold on the 90-frame median. Warn is one epoch's jitter; abort is the point where every sample-anchored window is measurably shorter than configured.
<code>rz2SkewWindowFrames</code>	90	<code>ConfigOrgParams.m</code> $\rightarrow$ <code>ClockSkewMonitor.m</code>	Frames in the sustained (median) window the abort is judged on (≈ 1.5 s at 60 fps).
<code>nonFiniteAbortRun</code>	10 $\times$ window	<code>ClockSkewMonitor.m</code> (constructor default)	Consecutive non-finite skew readings that trip the abort (≈ 900 frames, 15 s). Catches a clock map that never anchors because the queue never clears.
<code>rz2MaxSamplesPerDrain</code> (M)	2048	<code>ConfigOrgParams.m</code>	Per-frame drain ceiling. Governs backlog recovery time k^* (Eq. (10)); too large risks a long parse inside one frame, too small makes recovery slow. Lower it only if frames stutter while draining.
<code>rz2CapWarnFrames</code>	60	<code>ConfigOrgParams.m</code>	Consecutive capped frames before the <code>rz2:drainCapped</code> warning (≈ 1 s).

Parameter	Default	Location	Effect and guidance
rz2InputBufferSize	4 194 304 B	ConfigOrgParams.m	Socket receive buffer (and the OS receive buffer on the NIO channel). Must exceed the largest burst (≈ 29 B/sample).
rz2InputDatagram- PacketSize	8192 B	ConfigOrgParams.m	Largest datagram the legacy <code>udp()</code> object returns whole; its 512-byte default truncated every relay datagram. Load-bearing only on the legacy transport.
rz2UdpPort, rz2RelaySourcePort	8831, 8832	ConfigOrgParams.m	Must match the relay; the legacy object filters datagrams by remote host and port.
rz2ScaleX, rz2ScaleY (s_x, s_y)	1, 1	ConfigOrgParams.m, console “RZ2 gain”	Pixels per unit deflection as a fraction of W, H (Eq. (13)); sign inverts the axis. Changing them changes the CSV pixel coordinates, so the offline geometry must match the session.
rz2OffsetY (o_y)	0	ConfigOrgParams.m	Vertical rest-position correction in px.
joyGain	-1.3	console, ReadCursorPosition.m	USB-joystick counterpart of $s_x = s_y$.
moveOversample, moveOversampleDt	1, 0.008 s	CenterOutTask.m (USB and mouse only)	Extra reads per frame and their spacing; 8 ms matches the joystick’s measured HID update interval. Skipped entirely on <code>rz2adc</code> .
movementExportEpochs	{6, 7, 8}	CenterOutTask.m	Rows kept in <code>trajectory_movement_*.csv</code> . Removing 8 drops the hold flags offline.
trajChunk	200 000	CenterOutTask.m	Growth increment of <code>trajBuf</code> ; performance only.
Synapse downsample (D)	26	Synapse runtime parameter (Computer 1)	Sets $f_s = 24414.0625/D$ (Eq. (11)). Can reset to 1 on a Synapse restart; confirm at session start.
JOY_RANGE	4.6 V	InitJoystickRelay.m	Volts mapped to ± 1 ; values beyond are clipped. Changing it rescales every v and therefore every pixel coordinate.
RELAY_HZ	169.4	InitJoystickRelay.m	Relay cycle rate; sets datagrams/s and the per-cycle batch ($\approx f_s/169.4$). In write-index mode it does not set the read rate.
MAX_READ_PER_CYCLE, MAX_SAMPLES_PER_DGRAM	4096, 17	InitJoystickRelay.m	Burst ceiling per cycle, and lines per datagram (17×30 B worst case = 510 < 512).

Parameter	Default	Location	Effect and guidance
IDX_Y_CHECK_EVERY, IDX_Y_TOL	10, 2	InitJoystickRelay.m	Cadence of the Y write-index consistency read and the divergence that reverts to reading both every cycle.
BUF_SIZE (B)	100 000	InitJoystickRelay.m (must match the SerStore size)	Ring size; $B/f_s \approx 106$ s is how long Computer 1 may stall before samples are overwritten.
WRITE_IDX_TAG_X/Y	'???'	InitJoystickRelay.m	Non-empty enables write-index mode, but only after both names are found in <code>getParameterNames</code> on their gizmo at startup; a missing one warns and resets both to '' (§3.2). <code>N_READ</code> , <code>WRITE_FS</code> = 1017, <code>READ_FS</code> , <code>RECAL_*</code> serve only the legacy fallback, which is reachable but tuned to a write rate that was never real.

8.2 Offline chain (notebook, section 1.1 cell)

Parameter	Default	Location	Effect and guidance
hampel_half_window (h)	3	PipelineProfile → HampelScreen	Window $2h + 1$. At 939 Hz, 7 samples is ≈ 7.5 ms (58 ms on the USB path); a wider window tolerates longer dropouts but starts to treat fast real onsets as outliers.
hampel_n_sigma (n_σ)	3.0	same	Threshold in robust- σ units. Lower is more aggressive.
meas_sigma_px (σ_{meas})	2.0	PipelineProfile → KalmanTrajectory- Smoother	Measurement noise s.d. in px. Only the ratio q/R matters for the estimate.
jerk_sigma_px_s3 (σ_{jerk})	10^5	same	Process-noise s.d. of jerk; $q = \sigma_{\text{jerk}}^2$ in Eq. (24). Rough smoothing corner $(q/R)^{1/6}$: $10^5 \rightarrow 5.9$ Hz, $10^4 \rightarrow 2.7$ Hz, $10^6 \rightarrow 12.6$ Hz. Increasing it makes the stage a near pass-through.
gate_sigma (γ)	∞	same	χ^2 innovation gate. A finite value rejects measurements the model finds implausible; use with care, since a rejected update costs delays.
drop_unindexed_rows	True on rz2adc	PIPELINE_PROFILES	Removes <code>RZ2Idx = NaN</code> rows when the file is read (§6.1). Off on <code>usb</code> , where every row is NaN.

Parameter	Default	Location	Effect and guidance
RZ2_MAX_MEDIAN_-STEP_MS	3.0 ms	module constant	Threshold of the fallback source rule, Eq. (18). RZ2 delivers ≈ 1.06 ms steps, USB ≈ 8 ms.
INPUT_SOURCE_-OVERRIDES	{}	module constant	Maps a run tag (or part of one) to a source, bypassing Eq. (18).
grid_dt_s (Δ), GRID_DT_S	0.008	module constant, passed into TrajectoryProcessor	Resampling step (125 Hz). Also sets f_s for the Butterworth and Δ in the Savitzky–Golay derivative. Must satisfy $f_c < 1/(2\Delta)$.
CUTOFF_HZ (f_c)	6.0	module constant	Butterworth design cutoff; effective two-pass -3 dB at $0.802f_c$ (Eq. (38)). Note the class default is 20 Hz; the processor always overrides it with this constant.
EFFECTIVE_CUTOFF_HZ	4.82 Hz	computed	f_c mapped through the prewarped axis; used in figure titles so they stay correct when f_c or Δ change.
Butterworth-Lowpass.N_FACT (n_f)	6	class attribute	Reflection pad length; matches <code>filtfilt</code> 's default for a second-order section.
SAVGOL_WINDOW (w)	11	module constant	Derivative window in grid samples ($w\Delta = 88$ ms). Larger smooths acceleration more and lowers peaks. Must be odd.
SAVGOL_POLYORDER (p)	3	module constant	Local polynomial degree; must be $< w$.
ACCEL_PEAK_PERCENTILE	99.0	module constant	Percentile reported as peak acceleration.
TAKEOFF_SPEED_-FRAC_CHOICE	"5%"	module constant, <code>globals().get</code>	Selects ρ from <code>TAKEOFF_SPEED_FRAC_OPTIONS</code> ("5%" = 0.05, "10%" = 0.10) into <code>MOVE_SPEED_FRAC</code> . Sets the takeoff threshold, Eq. (41), and therefore the start of the summary window.
TAKEOFF_PEAK_CHOICE	"first"	module constant, <code>globals().get</code>	"first" walks back from the first prominent local speed peak; any other value walks back from the global maximum over $\mathcal{S}$. Only differs on trials with more than one submovement.
MULTI_PEAK_-PROMINENCE_FRAC	0.15	module constant, <code>globals().get</code>	Prominence a local maximum needs, as a fraction of the largest speed in $\mathcal{S}$, to count as a peak in <code>find_peaks</code> and therefore in <code>n_speed_peaks</code> . Lower it and ripple counts as a submovement.
min_move_samples, min_move_dur_s	5, None	TrajectoryProcessor	Under-resolved criteria, Eq. (43).

Parameter	Default	Location	Effect and guidance
PIXEL_PITCH_MM → CM_PER_PX (<i>c</i>)	0.3108 mm	module constants	Unit conversion for all cm quantities.
MOVE_EPOCHS	(6, 7, 8) or (6, 7)	module constant, per cell	Epoch codes loaded. The 1.1 cell uses (6, 7, 8) so the hold flags have rows; the trajectory-EDA cell and the feature table use (6, 7). The summary window is bounded by target entry, so the choice does not change the summaries.
INITIAL_HEADING_MS, MIN_HEADING_PX	120 ms, 12 px	change-of-mind cell	Window and minimum displacement for Eq. (48).
NOMINAL_BOUNDARIES	[5.05, 6.40]	chronometric cell	Fallback boundaries in deg VA; the fitted μ from 1.5 is used when available.
CENTER_DIAMETER_PX, TARGET_DIAMETER_PX, CENTER_TO_TARGET_DIST_PX, TARGET_DIST_SCALE	200, 180, 320, 1.27	module constants, overridden by <code>params_<runTag>.mat</code>	Geometry for the hold-instability threshold and the figure overlays. Now read from the session's own parameter snapshot when one exists (§6.1); the constants are the last fallback.
EARLY_EXIT_CODE	1	module constant, <code>globals().get</code>	<code>ErrorType</code> value that marks a trial ended without a target being chosen. Used twice: <code>run()</code> skips those trials and excludes them from the axis limits, and <code>drop_early_exits</code> removes them from the feature table.
MAX_TRAJ_TIME_MS	4000 ms	module constant	Hard upper limit of the shared trajectory time axis. Trials longer than this are drawn clipped and listed by Block/Trial/Attempt.
SHOW_K2_OVERLAY	True	clustering cell	When the silhouette picks $k \neq 2$, also fit $k = 2$ and cross-tabulate it against <code>IsCorrect</code> .
EXCLUDE_INVALID_SESSIONS, INVALID_SESSIONS	True, 2 entries	session loader	Sessions skipped with a printed reason (§6.9).

8.3 Recommended order of adjustment

1. Confirm f_s at session start (Synapse `downsample`) and read the teardown line `RZ2 clock: estimated write rate` together with the accepted/refused counts. A persistent gap between seed and estimate means `rz2SampleRateHz` should be updated to the measured value; “all refused” means a real backlog, not a clock problem.
2. Cross-check that estimate against the `OfflineClockFit` report for the same session, which lives in the single-session notebook (§6.9). Two estimators, one data set: agreement is evidence, disagreement localises the fault.
3. Keep σ_{jerk} and f_c on the same scale (they currently are, ≈ 6 Hz). If one is changed, revisit the other, and verify that content above $1/(2\Delta)$ is negligible before resampling.

4. Change w and p only together with Δ : the physical window is $w\Delta$.
5. The screen geometry is read from `params_<runTag>.mat`. Edit the module constants only for sessions that predate that export, or supply a `geometry_spec*.json` beside the trial data.

9 Observations and open points in the current code

These are properties of the version described above, found by reading it. They are not defects unless stated; several are deliberate choices whose consequences are worth having written down.

- **Consistent smoothing scales.** The Kalman defaults and the Butterworth cutoff both imply a corner near 6 Hz (Eq. (32) and Eq. (38)), so the pipeline is not over-filtering relative to its own stated bandwidth. The $(q/R)^{1/6}$ heuristic should be treated as an order of magnitude, not a specification.
- **The effective cutoff is 4.8 Hz, not 6 Hz.** Any report of the filter should quote `EFFECTIVE_CUTOFF_HZ`, which the code computes from f_c and the grid rate, rather than the design parameter.
- **The takeoff and the reported peak can belong to different submovements.** `move_takeoff_ms` is walked back from the first prominent peak (`TAKEOFF_PEAK_CHOICE = "first"`), while v_{peak} is the maximum over the whole window. On a single-submovement trial they coincide; on a corrected reach they do not, and that is deliberate. Any analysis that pairs a peak speed with an onset should carry `n_speed_peaks` alongside, and `peak_vel_time_ms` says which peak was reported.
- **The summary window is now one mask.** Peak, mean and median of both speed and acceleration are taken over $\mathcal{M}$ (5% takeoff to target entry, Eq. (42)). This removes the earlier asymmetry in which the peak was searched over the whole exported window while the means and medians were restricted to the movement epoch, and it is what makes the summaries independent of which epochs a given cell loads.
- **Duplicate timestamps are dropped keeping the first sample.** On the `rz2adc` path every duplicate is a cached row (write path 2), which carries the previous row's time by design and no new position, so dropping it is correct. The count is now reported per session in `df.attrs["n_duplicate_ts"]` and per trial in `TrialResult.n_duplicate_ts`. On the USB path a duplicate means the HID driver returned a cached report, which is also information-free.
- **The Hampel stage is inert during perfectly still holds** ($\text{MAD}_i = 0$ guard, §6.3). This fires more often on the USB path, whose quantisation step is 5–20 px, than on the RZ2 path, whose wire resolution is ≈ 0.2 px.
- **The clock map's floor latency is unmeasured.** It survives in b as a constant, cancels in every within-trial difference, and shifts every stimulus-to-response interval by that fixed amount. `DecisionTime_s` is therefore a reaction time plus an unknown constant, not plus a queue. Measuring the constant needs a hardware loopback.
- **The queue-clear gate trades coverage for correctness.** Only drains that emptied the queue feed the estimator (Eq. (4)). On a link that is chronically a few datagrams behind but stable, most observations are refused and the slope is estimated from the minority that cleared; the teardown accepted/refused counts say how thin that minority was. If it is zero, the seed stood in for the rate all session, and the report flags it. The non-finite abort run of §2.4 is the backstop for the case where it is zero for the whole session.

- **WRITE_FS = 1017 on Computer 1 is an open item, not a footnote.** It bends the “ms behind head” log line (7.7 % low) and `VerifyWidxWidy.m`’s reference (§3.2). The planned fix is measurement at relay startup rather than a new constant, so that the two machines produce independent estimates of the same quantity.
- **The write-index guard is a startup check, so a tag that dies mid-session is still not caught.** `InitJoystickRelay.m` now proves both tags exist with `getParameterNames` before write-index mode engages (§3.2), which closes the silent-freeze path a repointed tag used to open. It runs once, deliberately, to keep a fourth HTTP call out of the profiled cycle. The residual case, a tag that stops resolving after the session has started, remains undetectable through `getParameterValue`, which returns rather than throws; the symptom would be `Samples=0/Dgrams=0` in the 5-second log with no warning. That is a reading habit, not a code change, unless the failure ever appears.
- **The legacy pacing path is now reachable but still mis-tuned.** A missing tag drops the relay onto $\text{READ_FS} = \text{N_READ} \times \text{RELAY_HZ} = 1016.4 \text{ Hz}$ and the diff-peak recalibration, both set against a write rate that was never real ($\approx 939 \text{ Hz}$). The fallback is therefore a loud degradation, which is what it should be, and not a second correct mode. It inherits the `WRITE_FS` open item of the previous bullet.
- **RZ2Idx has two consumers offline, but only one of them in every notebook.** It selects the pipeline profile and drops the cached rows (§6.1) in both, and it supports the whole-session clock refit of §6.9 in the single-session notebook only. The multi-session notebook no longer runs that refit, so a session that is only ever seen in a multi-session sweep has its time base classified but not checked. The analyses still read `Time_ms` as the time base; the refit is a check on it. Re-deriving τ_i from the offline fit and re-running the pipeline on that time base remains possible and is not currently done.
- **The section 0.2 configuration cell is now inert.** It still sets `MOVE_SPEED_FRAC` and `MOVE_SPEED_FRAC_ALT` and prints both, but the signal-processing cells overwrite `MOVE_SPEED_FRAC` from `TAKEOFF_SPEED_FRAC_OPTIONS[TAKEOFF_SPEED_FRAC_CHOICE]` and no longer read the alternate fraction at all, so editing either value there changes nothing downstream while the printed line says it did. The threshold is now changed through `TAKEOFF_SPEED_FRAC_CHOICE`, and the cell’s own markdown (which still describes `move_takeoff_alt_ms` as a second reference line in the figures) describes a version that no longer exists. This is a documentation defect in the notebook, not in the pipeline, and it is the kind that silently costs an afternoon.
- **The offline geometry now comes from the session.** `ScreenGeometry.resolve` prefers `orgParams` in `params_<runTag>.mat`, so the notebook constants are a fallback rather than something an analyst must remember to edit. Sessions recorded before that snapshot existed still need the `geometry_spec*` override or the constants, and the printed line names which was used.
- **Epoch loading differs between cells by design.** The 1.1 cell loads (6,7,8) and the feature table loads (6,7). This no longer changes the kinematic summaries (see the two bullets on the summary window above), but it does change `hold_dur_ms`, `hold_max_speed_cm` and `hold_excursion_px`, which are only computable where the target-hold rows are present.
- **drop_early_exits is keyed on ErrorType, not on timing.** It removes `ErrorType = 1` and cross-checks against `ChosenTarget`. A negative `DecisionTime_s` is no longer a failure mode the current code can produce (§2.5), so nothing filters on it; if one ever appears in a new session it is a signal, not noise, and the monotonicity warning and the offline clock fit are where to look.